SISTEMAS AVANZADOS DE PRODUCCIÓN CON PYTHON

Objetivo general

Regresión y clasificación, utilizando una red neuronal.

Objetivos de la sesión

Clase 1: Ruta A — Problema de Regresión 🛠️

- En esta ruta seguiremos el ciclo completo de machine learning: análisis del problema, recolección de datos, exploración, y limpieza y preparación, para luego pasar al modelado y evaluación. El foco estará en predecir un valor continuo usando un baseline de regresión lineal y una red neuronal para regresión. Ten en cuenta que el orden y la profundidad de cada paso pueden variar según el dataset de cada grupo (por ejemplo, si ya viene limpio o si requiere más ingeniería de variables). Documentaremos claramente cada decisión tomada, las métricas usadas (MAE/RMSE) y las conclusiones.

Clase 2: Ruta B — Problema de Clasificación🌳

- Aquí aplicaremos el mismo ciclo: análisis del problema, recolección de datos, exploración, y limpieza y preparación, seguido de modelado y evaluación. Clasificaremos casos binarios comparando regresión logística, perceptrón y un árbol aleatorio (Random Forest). El flujo podrá ajustarse según las características de los datos de cada equipo (por ejemplo, manejo de desbalance o codificación de categorías). Dejaremos registradas las decisiones de preprocesamiento, la elección de la métrica (p. ej., F1/ROC-AUC) y la definición del umbral cuando corresponda.

Semana 8

🧭 Análisis del problema — Bike Sharing (UCI)

- **Problema de negocio (qué decisión habilita):** Predecir la **demanda diaria/horaria de bicicletas** para **planear capacidad** (flota, redistribución, turnos y mantenimiento), evitando **quedarse corto** (pérdida de ventas/servicio) o **sobredimensionar** (costos operativos altos).
- **Situación actual (contexto):** La empresa dispone de históricos de alquileres y clima, pero decide de forma **reactiva**. Se requiere una **estimación confiable** de la demanda para calendarizar personal y mover inventario (bicicletas) con anticipación.
- **Tarea de aprendizaje: Regresión supervisada.** Variable objetivo continua: `cnt` = número de alquileres (por **hora** o por **día**, según el archivo elegido).

Requisitos de datos (mínimos):

- Variables **exógenas (no estan influidas por ruido u otras variables)** disponibles antes de la decisión: clima (`temp`, `humidity`, `windspeed`, `weather`) y calendario (`season`, `holiday`, `workingday`, `hour/weekday`).
- **Calidad aceptable:** $<$5–10% de datos nulos manejables, tipificación correcta y **sin fuga de información**.

Fuentes de datos:

- **UCI Bike Sharing Dataset** (público): archivos `day.csv` / `hour.csv`.

  En este caso se puede acceder al recurso original, el cual es:

  https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset
- **Diccionario de variables** provisto en el repositorio original.

Variables Table

Variable Name	Role	Type	Description	Units	Missing Values
instant	ID	Integer	record index		no
dteday	Feature	Date	date		no
season	Feature	Categorical	1:winter, 2:spring, 3:summer, 4:fall		no
yr	Feature	Categorical	year (0: 2011, 1: 2012)		no
mnth	Feature	Categorical	month (1 to 12)		no
hr	Feature	Categorical	hour (0 to 23)		no
holiday	Feature	Binary	weather day is holiday or not (extracted from http://dchr.dc.gov/page/holiday-schedule)		no

Criterio de éxito (métrica + umbral de negocio):

- Métrica principal: **RMSE** (o **MAE** si hay outliers relevantes).
- Objetivo operativo (ejemplo): **RMSE ≤ 15 alquileres/hora** o **MAE ≤ 10**, consistente en test.

Riesgos y supuestos:

- **Estacionalidad/feriados** impactan fuertemente la demanda.
- **Cambios de política** o eventos atípicos pueden romper patrones.
- No se usan datos personales; dataset **agregado y no sensible**.

📥 Recolección de datos — Bike Sharing (UCI)

Usaremos el dataset público **UCI Bike Sharing**. Página del repositorio: https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset. Descarga directa (ZIP): https://archive.ics.uci.edu/ml/machine-learning-databases/00275/Bike-Sharing-Dataset.zip. Trabajaremos con **una sola granularidad** (día `day.csv` u hora `hour.csv`) y registraremos en el notebook el **enlace de origen** y la **fecha de descarga** para trazabilidad. Antes de modelar, haremos una verificación básica: tipos correctos, rangos razonables y nulos manejables; variables clave: objetivo `cnt`, clima (`temp`, `humidity`, `windspeed`, `weather`) y calendario (`season`, `holiday`, `workingday`, `weekday`, `hour` si aplica).

Aspectos legales y privacidad. Este dataset es agregado y no contiene identificadores personales, por lo que **no requiere anonimización adicional** en uso académico. Si algún equipo lo combina con datos propios, debe documentar la finalidad, eliminar

identificadores directos, agrupar o truncar campos sensibles y retener solo lo necesario, conforme a la **Ley 1581 de 2012 (Habeas Data)** en Colombia.

```
# ============================================================
# CARGA CORRECTA DEL DATASET "BIKE SHARING (UCI)" EN COLAB
# ------------------------------------------------------------
# Objetivo de esta celda:
# 1) Descargar el ZIP oficial desde UCI.
# 2) Descomprimirlo en el entorno de Colab.
# 3) Elegir la granularidad (día u hora) y cargar el CSV a pandas.
# 4) Convertir fechas a tipo datetime y revisar tipos de datos.
# 5) Mostrar un diccionario claro de variables (qué significa cada una).
# 6) Hacer verificaciones básicas: tamaño, nulos, muestra de filas.
# ------------------------------------------------------------
# Nota: Esta celda está MUY comentada a propósito, para personas
#       que están empezando con Python y Colab.
# ============================================================

import os
import io
import zipfile
import urllib.request
import pandas as pd

# ------------------------------------------------------------
# 1) CONFIGURACIÓN BÁSICA DEL PROYECTO (FÁCIL DE EDITAR)
# ------------------------------------------------------------

# Enlace oficial del ZIP en el repositorio UCI:
UCI_ZIP_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/00275/

# El usuario puede elegir la granularidad aquí:
#   - "day"  -> carga el archivo day.csv  (agregado por día)
#   - "hour" -> carga el archivo hour.csv (agregado por hora)
GRANULARITY = "hour"  # <-- Cambia a "day" si quieres trabajar por día

# Carpeta donde guardaremos los archivos descomprimidos
DATA_DIR = "/content/bike_sharing_data"

# ------------------------------------------------------------
# 2) DESCARGA DEL ZIP DESDE UCI
#    No requiere instalar nada extra. Usamos la librería estándar.
# ------------------------------------------------------------
os.makedirs(DATA_DIR, exist_ok=True)
zip_path = os.path.join(DATA_DIR, "Bike-Sharing-Dataset.zip")

print("Descargando ZIP desde UCI (puede tardar unos segundos)...")
urllib.request.urlretrieve(UCI_ZIP_URL, zip_path)
print("Descarga completa:", zip_path)
```

```
# ------------------------------------------------------------
# 3) DESCOMPRESIÓN DEL ZIP
# ------------------------------------------------------------
print("Descomprimiendo archivos...")
with zipfile.ZipFile(zip_path, 'r') as z:
    z.extractall(DATA_DIR)

print("Contenido descomprimido en:", DATA_DIR)
print("Archivos disponibles:", os.listdir(DATA_DIR))

# ------------------------------------------------------------
# 4) ELECCIÓN DEL CSV SEGÚN LA GRANULARIDAD
# ------------------------------------------------------------
if GRANULARITY == "day":
    csv_path = os.path.join(DATA_DIR, "day.csv")
elif GRANULARITY == "hour":
    csv_path = os.path.join(DATA_DIR, "hour.csv")
else:
    raise ValueError("GRANULARITY debe ser 'day' u 'hour'.")

if not os.path.exists(csv_path):
    raise FileNotFoundError(f"No se encontró el archivo esperado: {csv_path}")

print(f"Granularidad elegida: {GRANULARITY.upper()} -> Cargando {os.path.basena

# ------------------------------------------------------------
# 5) CARGA DEL CSV CON PANDAS
#    - parse_dates convierte la columna de fecha a tipo datetime (si existe).
#    - keep_default_na=True: pandas interpretará valores vacíos como NA.
# ------------------------------------------------------------
# En ambos CSV existe la columna 'dteday' (fecha). La parseamos si está present
parse_cols = ["dteday"]  # Si la columna no existe, pandas la ignorará.
df = pd.read_csv(csv_path, parse_dates=parse_cols, dayfirst=False, keep_default

# ------------------------------------------------------------
# 6) DICCIONARIO DE VARIABLES
#    Explicamos, en español y de forma clara, qué significa cada columna.
#    (Puede que en tu CSV haya leves diferencias; este diccionario cubre lo est
# ------------------------------------------------------------
variable_dict = {
    # Identificadores / fecha
    "instant": "Índice de la fila (identificador incremental).",
    "dteday": "Fecha del registro (tipo datetime).",
    "season": "Estación del año (1: invierno, 2: primavera, 3: verano, 4: otoño
    "yr": "Año (0: primer año del dataset, 1: segundo año).",
    "mnth": "Mes del año (1 a 12).",
    "hr": "Hora del día (0 a 23) — solo en hour.csv.",
    "holiday": "Indica si el día es festivo (1) o no (0).",
    "weekday": "Día de la semana (0: domingo… 6: sábado).",
    "workingday": "Día laboral (1) o no (0) — excluye fines de semana y festivo
    # Clima
```

```
    "weathersit": "Situación del clima (1: bueno, 2: nubes/neblina, 3: lluvia l
    "temp": "Temperatura normalizada (0 a 1) — basada en temperatura en °C.",
    "atemp": "Temperatura aparente normalizada (sensación térmica).",
    "hum": "Humedad normalizada (0 a 1).",
    "windspeed": "Velocidad del viento normalizada (0 a 1).",
    # Objetivo y componentes
    "casual": "Número de alquileres por usuarios casuales (no registrados).",
    "registered": "Número de alquileres por usuarios registrados.",
    "cnt": "Número total de alquileres (objetivo para regresión)."
}

# Mostramos el diccionario como una tabla legible
dict_df = pd.DataFrame(
    [(k, variable_dict.get(k, "(desconocida en este diccionario)")) for k in df
    columns=["columna", "significado"]
)

print("\n=== DICCIONARIO DE VARIABLES (según columnas presentes) ===")
print(dict_df.to_string(index=False))

# ------------------------------------------------------------
# 7) ARREGLOS Y TIPOS DE DATOS
#    - Confirmamos que 'dteday' está en datetime.
#    - Revisamos tipos básicos y rangos.
# ------------------------------------------------------------
if "dteday" in df.columns:
    # Ya la parseamos como datetime en read_csv; confirmamos:
    print("\nTipo de 'dteday':", df["dteday"].dtype)

print("\n=== TIPOS DE DATOS (pandas dtypes) ===")
print(df.dtypes)

# ------------------------------------------------------------
# 8) VERIFICACIONES BÁSICAS DE CALIDAD
# ------------------------------------------------------------
print("\n=== DIMENSIONES DEL DATASET ===")
print("Filas:", len(df), "| Columnas:", df.shape[1])

print("\n=== NULOS POR COLUMNA (valores faltantes) ===")
print(df.isna().sum())



# ------------------------------------------------------------
# 9) SUGERENCIA PEDAGÓGICA
#    Si estamos en 'hour.csv', a menudo es útil crear una columna 'hour' si no
#    o asegurar que 'hr' está en entero. También es buena práctica comprobar va
# ------------------------------------------------------------
if GRANULARITY == "hour":
    # Aseguramos que hr sea entero (debería serlo ya).
    if "hr" in df.columns:
```

```
        df["hr"] = df["hr"].astype(int)
        print("\nValores únicos de 'hr' (deberían ser 0..23):", sorted(df["hr"]

# ------------------------------------------------------------
# 10) (OPCIONAL) GUARDAR UNA COPIA LIMPIA INICIAL
#     Esto ayuda a mantener una "versión base" antes de empezar la preparación.
# ------------------------------------------------------------
base_csv_path = os.path.join(DATA_DIR, f"base_{GRANULARITY}.csv")
df.to_csv(base_csv_path, index=False)
print(f"\nCopia base guardada en: {base_csv_path}")

# Listo: en las siguientes celdas se recomienda continuar con:
# - EXPLORACIÓN (estadísticos, gráficos básicos)
# - LIMPIEZA/PREPARACIÓN (nulos, tipos, escalado, codificación)
# - PARTICIÓN (train/test)
# - MODELADO (Regresión Lineal vs Perceptrón/Adaline/MLP)
# - EVALUACIÓN (MAE/RMSE y residuales)
```

```
Descargando ZIP desde UCI (puede tardar unos segundos)...
Descarga completa: /content/bike_sharing_data/Bike-Sharing-Dataset.zip
Descomprimiendo archivos...
Contenido descomprimido en: /content/bike_sharing_data
Archivos disponibles: ['Readme.txt', 'day.csv', 'Bike-Sharing-Dataset.zip', 'h
Granularidad elegida: HOUR -> Cargando hour.csv

=== DICCIONARIO DE VARIABLES (según columnas presentes) ===
   columna
   instant                                                Índice de la fila (ide
    dteday                                                           Fecha del r
    season        Estación del año (1: invierno, 2: primavera, 3: verano, 4: o
        yr                                            Año (0: primer año del d
      mnth
        hr                                                     Hora del día (0 a
   holiday                                                     Indica si el día
   weekday                                                     Día de la semana
workingday                                Día laboral (1) o no (0) — excluye fin
weathersit Situación del clima (1: bueno, 2: nubes/neblina, 3: lluvia ligera, 
      temp                                Temperatura normalizada (0 a 1) — basa
     atemp                                         Temperatura aparente normaliz
       hum                                                                 Hume
 windspeed                                                     Velocidad del vie
    casual                                  Número de alquileres por usuarios ca
registered                                                Número de alquileres p
       cnt                                         Número total de alquileres (o

Tipo de 'dteday': datetime64[ns]

=== TIPOS DE DATOS (pandas dtypes) ===
instant                int64
dteday        datetime64[ns]
season                 int64
yr                     int64
mnth                   int64
hr                     int64
```

```
holiday                int64
weekday                int64
workingday             int64
weathersit             int64
temp                 float64
atemp                float64
hum                  float64
windspeed            float64
casual                 int64
registered             int64
cnt                    int64
dtype: object

=== DIMENSIONES DEL DATASET ===
Filas: 17379 | Columnas: 17

=== NULOS POR COLUMNA (valores faltantes) ===
instant       0
dteday        0
season        0
yr            0
```

```
# Vista del data frame
print("\n=== VISTA RÁPIDA (head) ===")
df.head()
```

```
=== VISTA RÁPIDA (head) ===
```

	instant	dteday	season	yr	mnth	hr	holiday	weekday	workingday	weathersi
0	1	2011-01-01	1	0	1	0	0	6	0	
1	2	2011-01-01	1	0	1	1	0	6	0	
2	3	2011-01-01	1	0	1	2	0	6	0	
3	4	2011-01-01	1	0	1	3	0	6	0	
4	5	2011-01-01	1	0	1	4	0	6	0	

🔍 Exploración y conocimiento de los datos — Bike Sharing (UCI)

En esta fase buscamos **entender la forma y el contexto** del dataset antes de modelar. Revisamos tamaño (filas/columnas), tipos de variables, rangos y valores atípicos, y

verificamos si la granularidad elegida (**día** u **hora**) cubre estacionalidad y feriados. La idea es detectar patrones generales (tendencias por clima y calendario), posibles **nulos** o codificaciones inusuales, y confirmar que la variable objetivo `cnt` refleja correctamente la demanda.

Descripción breve. Empezamos con una síntesis: periodo cubierto, frecuencia (día/hora), medias y dispersiones de las variables numéricas, y distribución básica de `cnt`. Si existen transformaciones previas (p. ej., temperaturas normalizadas 0–1), se anotan para no duplicarlas. Si hay desbalance temporal (pocos registros en ciertos meses/horas), se documenta porque afectará la validación.

📚 Diccionario de datos (significado de variables)

Variable	Significado breve
`instant`	Índice incremental del registro.
`dteday`	Fecha del registro (día).
`season`	Estación (1 invierno, 2 primavera, 3 verano, 4 otoño).
`yr`	Año (0 o 1 según el año en el dataset).
`mnth`	Mes (1–12).
`hr`	Hora del día (0–23, solo en `hour.csv`).
`holiday`	Indicador de festivo (1 sí, 0 no).
`weekday`	Día de la semana (0 domingo … 6 sábado).
`workingday`	Día laboral (1 sí, 0 no; excluye fines de semana y festivos).
`weathersit`	Clima (1 bueno, 2 nublado/neblina, 3 lluvia ligera, 4 lluvia fuerte/nieve).
`temp`	Temperatura normalizada (0–1).
`atemp`	Sensación térmica normalizada (0–1).
`hum`	Humedad relativa normalizada (0–1).
`windspeed`	Velocidad del viento normalizada (0–1).
`casual`	Alquileres de usuarios casuales (no registrados).
`registered`	Alquileres de usuarios registrados.
`cnt`	**Objetivo**: total de alquileres (`casual + registered`).

> Nota: algunas columnas son **categóricas codificadas como números** (p. ej., `season`, `weathersit`, `weekday`). Se interpretan como categorías, no como cantidades continuas.

🖼️ Guía rápida para gráficos descriptivos

Histogramas sirven para ver la **distribución** de una variable numérica (p. ej., `cnt`, `temp`, `hum`) y detectar asimetrías o multimodalidad. Úsalos cuando quieras responder

"¿cómo se reparte esta variable?" o comparar distribuciones por subgrupos (ej., histograma de `cnt` por `season`).

Box plots son útiles para comparar **niveles y dispersión** entre **categorías** y detectar **outliers** (p. ej., `cnt` por `weekday`, `weathersit` o `hour`). Son ideales para responder "¿qué categoría tiende a tener valores más altos/estables?" y visualizar mediana y cuartiles.

Series/line plots (si trabajas por hora o día) muestran **tendencias y estacionalidad** en el tiempo (picos por hora punta, patrones de fin de semana). Ayudan a ver ciclos diarios/semanales y posibles quiebres.

Relaciones bivariadas: un **scatter** `temp` vs `cnt` sugiere relación con clima; un **scatter** `hr` vs `cnt` revela picos horarios. Si aparece curvatura o bandas, puede ser señal de **no linealidad** o de variables omitidas.

Qué dejamos documentado.

Un párrafo con los **hallazgos descriptivos clave**: patrón de demanda por hora/día, efecto visible del clima y presencia de outliers o nulos. Además, reportamos **correlaciones** entre `cnt` y las variables numéricas con los coeficientes **de Pearson** (relación **lineal**) y **de Spearman** (relación **monótona**, robusta a no linealidades y outliers). Si Pearson es alto y Spearman similar, sugiere relación aproximadamente lineal; si Spearman es alto pero Pearson bajo, puede haber relación no lineal (o influencia de rangos). Esta evidencia guía la **preparación** (p. ej., transformaciones/escala) y anticipa si una **regresión lineal** podría bastar o si un **modelo no lineal** agregará valor.

```
!pip -q install ydata-profiling -q
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 400.4/400.4 kB 7.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 296.7/296.7 kB 22.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.1/3.1 MB 51.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 679.7/679.7 kB 23.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 105.4/105.4 kB 4.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 67.9/67.9 kB 2.9 MB/s eta 0:00:00
^C
```

```
# Utilizamos la libreria recien instalada
from ydata_profiling import ProfileReport

profile = ProfileReport(df)
profile.to_notebook_iframe()  # lo embebe como iframe (funciona bien en Colab)
# Descarga el profile como archivo html
profile.to_file("reporte_perfil.html")
```

```
print("Archivo guardado:", "/content/reporte_perfil.html")
```

```
# Camino mas largo, pero enriqucedor para conocer otro tipo de gráficos.
# ==============================================================
# EXPLORACIÓN DESCRIPTIVA — UCI Bike Sharing (day/hour)
# --------------------------------------------------------------
# Qué hace esta celda (paso a paso):
# 1) Grafica histogramas (forma de la distribución).
# 2) Grafica boxplots por categorías (estación, clima, etc.).
# 3) Grafica tendencias (promedios por hora / weekday).
# 4) Grafica relaciones (scatter) cnt vs temp / hr.
# 5) Calcula correlación Pearson y Spearman y dibuja heatmaps.
# --------------------------------------------------------------
# Requisitos: SOLO matplotlib y pandas (ya vienen en Colab).
# ==============================================================

# ========== 0) CONFIGURACIÓN DEL PROYECTO ==========
GRANULARITY = "hour"  # Cambia a "day" para trabajar por día
PRIMARY_TARGET = "cnt"  # Variable objetivo de demanda total

# ========== 1) DESCARGA Y CARGA ==========
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.express as px

# ========== 1) HISTOGRAMAS ==========
# Qué vemos: forma de la distribución (asimetría, colas, multimodalidad)
numeric_cols = [c for c in ["cnt","temp","atemp","hum","windspeed"] if c in df.
for col in numeric_cols:
    plt.figure(figsize=(7,4))
    plt.hist(df[col].dropna(), bins=30)  # No definimos color explícito
    plt.title(f"Histograma de {col}")
    plt.xlabel(col)
    plt.ylabel("Frecuencia")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

# ========== 2) BOXPLOTS POR CATEGORÍAS ==========
# Qué vemos: diferencias de nivel y dispersión entre categorías (y outliers)
def boxplot_by_category(df, cat_col, y_col=PRIMARY_TARGET):
    if cat_col not in df.columns or y_col not in df.columns:
        return
    data = [g[y_col].values for _, g in df.groupby(cat_col)]
    labels = [str(lbl) for lbl in sorted(df[cat_col].unique())]
    plt.figure(figsize=(8,4))
    plt.boxplot(data, labels=labels, showmeans=True)
```

```
    plt.title(f"{y_col} por {cat_col}")
    plt.xlabel(cat_col)
    plt.ylabel(y_col)
    plt.grid(True, axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

for cat in ["season", "weathersit", "weekday"] + (["hr"] if GRANULARITY=="hour"
    boxplot_by_category(df, cat, PRIMARY_TARGET)

# ========== 3) SERIES / TENDENCIAS ==========
# Qué vemos: patrones por hora y por día de la semana (picos, estacionalidad cc
if GRANULARITY == "hour" and "hr" in df.columns:
    avg_by_hour = df.groupby("hr")[PRIMARY_TARGET].mean()
    plt.figure(figsize=(8,4))
    plt.plot(avg_by_hour.index, avg_by_hour.values, marker="o")  # sin color ex
    plt.title("Promedio de alquileres por hora (cnt)")
    plt.xlabel("hr (0–23)")
    plt.ylabel("cnt (promedio)")
    plt.grid(True, alpha=0.3)
    plt.xticks(range(0,24,2))
    plt.tight_layout()
    plt.show()

avg_by_weekday = df.groupby("weekday")[PRIMARY_TARGET].mean()
plt.figure(figsize=(8,4))
plt.plot(avg_by_weekday.index, avg_by_weekday.values, marker="o")
plt.title("Promedio de alquileres por día de la semana (cnt)")
plt.xlabel("weekday (0=dom ... 6=sáb)")
plt.ylabel("cnt (promedio)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ========== 4) SCATTER PLOTS (relaciones) ==========
# Qué vemos: relación bivariada y posibles no linealidades
def scatter_basic(x, y, x_label, y_label):
    plt.figure(figsize=(7,4))
    plt.scatter(x, y, s=10, alpha=0.4)  # sin color específico
    plt.title(f"{y_label} vs {x_label}")
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
# Detectar columnas numéricas y excluir el target
num_cols_all = df.select_dtypes(include=[np.number]).columns.tolist()
num_features = [c for c in num_cols_all if c != PRIMARY_TARGET]

# Generar un scatter por cada variable numérica vs cnt
for col in num_features:
```

```
    scatter_basic(df[col], df[PRIMARY_TARGET], f"{col}", f"{PRIMARY_TARGET} (al

if "temp" in df.columns:
    scatter_basic(df["temp"], df[PRIMARY_TARGET], "temp (normalizada)", "cnt (a

if GRANULARITY == "hour" and "hr" in df.columns:
    scatter_basic(df["hr"], df[PRIMARY_TARGET], "hr (hora del día)", "cnt (alqu

# ========== 5) CORRELACIÓN PEARSON & SPEARMAN + HEATMAPS ==========
# Pearson: relación lineal. Spearman: relación monótona (por rangos; robusta a
num_df = df.select_dtypes(include=[np.number])

pearson_corr = num_df.corr(method="pearson")
spearman_corr = num_df.corr(method="spearman")

print("\n=== Correlación (Pearson) — top con cnt ===")
if PRIMARY_TARGET in pearson_corr.columns:
    print(pearson_corr[PRIMARY_TARGET].sort_values(ascending=False))

print("\n=== Correlación (Spearman) — top con cnt ===")
if PRIMARY_TARGET in spearman_corr.columns:
    print(spearman_corr[PRIMARY_TARGET].sort_values(ascending=False))

def heatmap_corr(corr_matrix, title, cmap="coolwarm"):
    # IMPORTANTE: se usa un colormap explícito porque el usuario lo pidió
    data = corr_matrix.values
    labels = corr_matrix.columns.tolist()
    n = data.shape[0]

    plt.figure(figsize=(max(6, n*0.5), max(5, n*0.5)))
    im = plt.imshow(data, cmap=cmap, vmin=-1, vmax=1)
    plt.title(title)
    cbar = plt.colorbar(im, fraction=0.046, pad=0.04)
    cbar.set_label("Coeficiente de correlación", rotation=90)

    plt.xticks(range(n), labels, rotation=90)
    plt.yticks(range(n), labels)

    # Anotar valores numéricos en cada celda
    for i in range(n):
        for j in range(n):
            plt.text(j, i, f"{data[i,j]:.2f}", ha="center", va="center", fontsi

    plt.tight_layout()
    plt.show()

heatmap_corr(pearson_corr, "Matriz de correlación (Pearson) — variables numéric
heatmap_corr(spearman_corr, "Matriz de correlación (Spearman) — variables numér


# ====== 6) Categóricas (PIE y SUNBURST) ======
```

```
# Pie simple por 'season' (proporción de registros)
if "season" in df.columns:
    tmp = df["season"].value_counts(normalize=True).reset_index()
    tmp.columns = ["season", "proporcion"]
    fig = px.pie(tmp, names="season", values="proporcion", title="Distribución
    fig.show()

# Sunburst ponderado por cnt: season -> weathersit (más útil que solo contar fi
if all(c in df.columns for c in ["season","weathersit", PRIMARY_TARGET]):
    tmp2 = df.groupby(["season","weathersit"], as_index=False)[PRIMARY_TARGET].
    fig2 = px.sunburst(tmp2, path=["season","weathersit"], values=PRIMARY_TARGE
                       title=f"Sunburst de {PRIMARY_TARGET} por season y weathe
    fig2.show()


# ========== 7) MINI-GUÍA (interpretación) ==========
print("\nGuía de lectura:")
print("- Histogramas: forma de cada variable (asimetría, colas, multimodalidad)
print("- Box plots: diferencias entre categorías (estación, clima, días) y outl
print("- Series: picos horarios y patrón por weekday (si trabajas por hora).")
print("- Scatter: relaciones y posibles no linealidades (temp/hr vs cnt).")
print("- Correlación: Pearson (lineal) y Spearman (monótona).")
print("  Si Pearson y Spearman son similares y altos con cnt -> tendencia linea
print("  si Spearman es alto y Pearson bajo -> relación no lineal o efecto de r
```

🧼 Limpieza y preparación — Bike Sharing (UCI)

Primero validamos **consistencia y tipos**:

`dteday` debe ser fecha; `season`, `weathersit`, `weekday`, `holiday`, `workingday` son **categóricas codificadas como números** (no continuas) y se tratarán como tales. Verificamos rangos razonables en `temp`, `atemp`, `hum`, `windspeed` (0–1 normalizados) y que **`cnt = casual + registered`** se cumpla; si no, corregimos o anotamos el caso y lo excluimos del modelado. También buscamos **duplicados** exactos por clave temporal (día u hora) y los removemos si no son intencionales.

Para **errores y valores atípicos**:

revisamos picos imposibles (p. ej., `windspeed=0` con tormenta severa) y fechas fuera de rango. Los outliers operativos se abordan según su origen: si son reales (eventos/feriados), se mantienen; si son errores, se **capan** (winsorizing) o se corrigen con reglas simples documentadas. Evitamos transformar la variable objetivo sin justificación; si la distribución de `cnt` es muy asimétrica, se puede evaluar una **transformación log(1+cnt)** solo si mejora la interpretación y la métrica.

El **tratamiento de datos faltantes** depende del patrón:

si el porcentaje es bajo y aleatorio, usamos **imputación simple** (mediana en numéricas, categoría "desconocido" en categóricas). Si los faltantes se concentran en ciertos periodos o categorías, mejor es **eliminar** esos registros si no compromete la cobertura estacional; de lo contrario, imputar y **marcar con una bandera** ("is_missing") para que el modelo capte el patrón.

La **ingeniería de características** parte de `dteday`:

extraemos **mes (`mnth`)**, **día de la semana (`weekday`)** y, en granularidad horaria, **hora (`hr`)**; añadimos indicadores como **fin de semana** y proximidad a feriados si aporta. Las variables categóricas (`season`, `weathersit`, `weekday`, `holiday`, `workingday`, y `hr` si aplica) se codifican con **One-Hot Encoding** para que los modelos lineales y redes interpreten correctamente sus efectos sin imponer un orden artificial. Evitamos crear variables que miren el futuro (p. ej., promedios que incluyan el propio registro) para **prevenir fuga de información**.

En cuanto a **escalado**, los modelos sensibles a la magnitud (Perceptrón/Adaline y MLP) requieren **estandarización** de las variables numéricas; Regresión Lineal también se beneficia al mejorar estabilidad numérica y comparabilidad de coeficientes (si hay regularización). El escalado, la codificación y cualquier imputación deben ir dentro de un **pipeline** para que lo mismo que se aprende en `train` se aplique en `valid` de forma consistente.

Finalmente, definimos la **partición de datos** antes de cualquier cálculo de promedios o transformaciones que usen información global. Si trabajamos con datos por hora o día, preferimos **partición temporal** (entrenar en el pasado, validar en el futuro) para evaluar generalización real; si el objetivo es puramente estático, puede usarse un `train/valid` aleatorio con semilla fija, dejando constancia de la elección y su impacto en la interpretación de resultados.

```
# ============================================================
# LIMPIEZA Y PREPARACIÓN (Bike Sharing UCI) — VERSIÓN SENCILLA
# ------------------------------------------------------------
# ¿Qué hace esta celda?
# 1) Valida y copia el DataFrame df.
# 2) Crea features útiles (is_weekend, is_peak_hour).
# 3) Define columnas numéricas y categóricas (sin fuga de información).
# 4) Parte en TRAIN / TEST (aleatorio por defecto; opcional temporal).
# 5) Imputa faltantes (mediana en numéricas, 'desconocido' en categóricas).
# 6) Aplica ONE-HOT a categóricas **entrenando en TRAIN** y alineando TEST.
# 7) Escala solo numéricas con StandardScaler (ajuste en TRAIN; aplica en TEST)
# 8) Imprime un breve resumen de lo obtenido.
# ============================================================
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler,OneHotEncoder

# ------------------------------
# 0) VALIDACIONES BÁSICAS
# ------------------------------
if 'df' not in globals():
    raise RuntimeError("No encuentro el DataFrame `df`. Corre antes la celda de

data = df.copy()                   # trabajamos sobre copia
TARGET = "cnt"                     # variable objetivo (total de alquileres)
assert TARGET in data.columns, f"'{TARGET}' no está en las columnas."

IS_HOURLY = 'hr' in data.columns  # granularidad (horaria si existe 'hr')

# Aseguramos que la fecha sea datetime (útil si hacemos split temporal)
if 'dteday' in data.columns and not np.issubdtype(data['dteday'].dtype, np.date
    data['dteday'] = pd.to_datetime(data['dteday'], errors='coerce')

# ------------------------------
# 1) INGENIERÍA DE CARACTERÍSTICAS SIMPLE (opcional y didáctica)
# ------------------------------
# (a) Fin de semana: weekday (0=dom, ... , 6=sáb) -> 1 si dom/sáb, 0 si no
if 'weekday' in data.columns:
    data['is_weekend'] = data['weekday'].isin([0,6]).astype(int)

# (b) Hora pico (si es horaria): 7-9 y 17-19
if IS_HOURLY and 'hr' in data.columns:
    data['is_peak_hour'] = data['hr'].isin([7,8,9,17,18,19]).astype(int)

# ------------------------------
# 2) DEFINIR QUÉ FEATURES USAR (¡evitamos fuga!)
# ------------------------------
# Evitamos columnas que causarían fuga o no aportan:
#  - 'casual' y 'registered' suman el target --> NO usarlas como features
#  - 'instant' es solo un índice
#  - 'dteday' (fecha cruda) no la modelamos directa; si quieres mes/semana, der
exclude = {'instant', 'casual', 'registered', TARGET}

# Numéricas continuas típicas del dataset (normalizadas [0,1])
num_cols = [c for c in ['temp','atemp','hum','windspeed'] if c in data.columns]

# Categóricas codificadas como números que trataremos como "categoría"
cat_cols = [c for c in ['season','weathersit','weekday','holiday','workingday',
if IS_HOURLY and 'hr' in data.columns:
    cat_cols.append('hr')

# Features adicionales que pudimos crear arriba
extra_cols = [c for c in ['is_weekend','is_peak_hour'] if c in data.columns]
```

```
# Conjunto final de columnas a usar
feature_cols = [c for c in (num_cols + cat_cols + extra_cols) if c not in exclu

# X (features) e y (target) antes del split
X_all = data[feature_cols].copy()
y_all = data[TARGET].copy()

# Convertimos las categóricas a tipo category (buena práctica)
for c in cat_cols:
    if c in X_all.columns:
        X_all[c] = X_all[c].astype('category')

# ------------------------------
# 3) PARTICIÓN TRAIN / TEST
# ------------------------------
SPLIT_METHOD = "random"  # "random" (por defecto) o "time" (temporal)
TEST_SIZE = 0.2          # 80% train / 20% test

if SPLIT_METHOD == "time" and 'dteday' in data.columns:
    # Ordenamos por tiempo; los últimos % serán TEST
    order_cols = ['dteday'] + (['hr'] if IS_HOURLY else [])
    sorted_idx = data.sort_values(order_cols).index
    split_point = int(len(sorted_idx) * (1 - TEST_SIZE))
    train_idx, test_idx = sorted_idx[:split_point], sorted_idx[split_point:]
    X_train, X_test = X_all.loc[train_idx].copy(), X_all.loc[test_idx].copy()
    y_train, y_test = y_all.loc[train_idx].copy(), y_all.loc[test_idx].copy()
else:
    # Aleatorio reproducible
    X_train, X_test, y_train, y_test = train_test_split(
        X_all, y_all, test_size=TEST_SIZE, random_state=42, shuffle=True
    )

print(f"[Split] Train: {len(X_train)} | Test: {len(X_test)} | Método: {SPLIT_ME

# ------------------------------
# 4) IMPUTACIÓN DE FALTANTES (simple y sin fuga)
# ------------------------------
# Numéricas -> mediana; Categóricas -> 'desconocido'
num_in_train = [c for c in num_cols if c in X_train.columns]
cat_in_train = [c for c in cat_cols if c in X_train.columns]

# Imputación numérica
if num_in_train:
    medians = X_train[num_in_train].median(numeric_only=True)
    X_train[num_in_train] = X_train[num_in_train].fillna(medians)
    X_test[num_in_train]  = X_test[num_in_train].fillna(medians)

# Imputación categórica
for c in cat_in_train:
    X_train[c] = X_train[c].astype('category')
```

```
        X_test[c]  = X_test[c].astype('category')
        # agregamos categoría 'desconocido' si hace falta y rellenamos NaN
        if 'desconocido' not in X_train[c].cat.categories:
            X_train[c] = X_train[c].cat.add_categories(['desconocido'])
        if 'desconocido' not in X_test[c].cat.categories:
            X_test[c] = X_test[c].cat.add_categories(['desconocido'])
        X_train[c] = X_train[c].fillna('desconocido')
        X_test[c]  = X_test[c].fillna('desconocido')

    # ------------------------------
    # 5) ONE-HOT ENCODING (sin fuga de información)
    # ------------------------------
    # *** ¿Qué es "one-hot"? ***
    #   Convertimos cada columna categórica en varias columnas binarias (0/1),
    #   una por categoría. Ej: 'season' con valores {1,2,3,4} genera:
    #     season_2, season_3, season_4  (si drop_first=True)  -> evita colinealidac
    # *** ¿Por qué "sin fuga"? ***
    #   - Ajustamos categorías SOLO con TRAIN (get_dummies en TRAIN).
    #   - Luego, en TEST reindexamos para tener EXACTAMENTE las mismas columnas;
    #     si falta alguna, rellenamos con 0.
    X_train_cat = pd.get_dummies(X_train[cat_in_train], drop_first=True) if cat_in_
    X_test_cat  = pd.get_dummies(X_test[cat_in_train],  drop_first=True) if cat_in_

    # Alinear columnas (clave para no introducir fuga / errores de dimensión)
    X_test_cat = X_test_cat.reindex(columns=X_train_cat.columns, fill_value=0)

    # Concatenamos con las numéricas originales (ya imputadas)
    X_train_num = X_train[num_in_train].copy()
    X_test_num  = X_test[num_in_train].copy()

    X_train_pre = pd.concat([X_train_num, X_train_cat], axis=1)
    X_test_pre  = pd.concat([X_test_num,  X_test_cat],  axis=1)

    # ------------------------------
    # 6) ESCALADO DE NUMÉRICAS (opcional pero recomendado para MLP/Perceptrón)
    # ------------------------------
    # Usamos StandardScaler: media=0, desvío=1
    scaler = None
    if num_in_train:
        scaler = StandardScaler()
        scaler.fit(X_train_pre[num_in_train])                          # ajusta SC
        X_train_pre[num_in_train] = scaler.transform(X_train_pre[num_in_train])
        X_test_pre[num_in_train]  = scaler.transform(X_test_pre[num_in_train])

    # ------------------------------
    # 7) RESUMEN FINAL
    # ------------------------------
    print("\n=== RESUMEN DE PREPARACIÓN ===")
    print(f"- Numéricas usadas: {num_in_train}")
    print(f"- Categóricas usadas (one-hot aplicado): {cat_in_train}")
    print(f"- Dimensiones -> X_train_pre: {X_train_pre.shape} | X_test_pre: {X_test
```

```
print("- Listos para modelar:\n"
      "  * Regresión Lineal (baseline)\n"
      "  * Perceptrón/Adaline/MLP (sensible a escala)\n"
      "  * Árboles/RandomForest/GBM (no requieren escalado)\n")
```

```
# ============================================================
# VISTA "ANTES vs DESPUÉS" EN UNA PROPORCIÓN DE FILAS
# ------------------------------------------------------------
# Objetivo:
# - Mostrar una muestra (proporción) del dataset ORIGINAL (antes)
#   y de los datos PREPARADOS (después) para comparar.
# - Ver formas (shape), columnas y ejemplos lado a lado.
#
# Requisitos:
# - Haber corrido las celdas anteriores:
#   * CARGA DE DATOS (crea `df`)
#   * LIMPIEZA Y PREPARACIÓN (crea `X_train_pre`, `X_valid_pre`, `y_train`, `y_
#     y variables auxiliares como `feature_cols`, `target_col`).
# ============================================================

import pandas as pd
import numpy as np

# --------- Validaciones mínimas ---------
needed_vars = ["df", "X_train_pre", "X_test_pre", "y_train", "y_test"]
for v in needed_vars:
    if v not in globals():
        raise RuntimeError(f"Falta la variable `{v}`. Por favor, ejecuta antes

# Parámetros de muestreo (puedes ajustarlos):
FRACTION = 0.02    # proporción de filas a mostrar (2%). Si el dataset es muy g
MIN_ROWS = 10       # al menos estas filas
MAX_ROWS = 40       # como máximo estas filas (para no saturar la vista)
RANDOM_STATE = 42

# --------- Construimos "ANTES" (original) ---------
# Usamos solo columnas relevantes: features + target (si existen)
target_col = 'cnt' if 'target_col' not in globals() else target_col
if 'feature_cols' not in globals():
    # Si no está definida (caso excepcional), intentamos una inferencia simple
    # tomando columnas no prohibidas como features.
    exclude_cols = set(['instant','dteday','casual','registered', target_col])
    # Numéricas y categóricas típicas en Bike Sharing
    candidate_cols = [c for c in df.columns if c not in exclude_cols]
    feature_cols_view = candidate_cols
else:
    feature_cols_view = feature_cols

cols_before = [c for c in feature_cols_view if c in df.columns] + ([target_col]
```

```
df_before = df[cols_before].copy()

# --------- Construimos "DESPUÉS" (preparado) ---------
# Unimos X_train_pre y y_train (lo mismo para valid) para mostrar ejemplos.
train_prepared = X_train_pre.copy()
train_prepared[target_col] = y_train.values

valid_prepared = X_test_pre.copy()
valid_prepared[target_col] = y_test.values

# Concatenamos train+valid preparados para una vista global "después"
df_after = pd.concat([train_prepared.assign(_split="train"),
                      valid_prepared.assign(_split="valid")], axis=0, ignore_ir

# --------- Muestreos controlados (misma cantidad antes y después) ---------
def choose_n_rows(total_len, frac, min_rows, max_rows):
    n = int(np.ceil(total_len * frac))
    n = max(min_rows, n)
    n = min(max_rows, n)
    n = min(n, total_len)  # no exceder el total
    return n

n_before = choose_n_rows(len(df_before), FRACTION, MIN_ROWS, MAX_ROWS)
n_after  = choose_n_rows(len(df_after), FRACTION, MIN_ROWS, MAX_ROWS)

sample_before = df_before.sample(n=n_before, random_state=RANDOM_STATE)
sample_after  = df_after.sample(n=n_after, random_state=RANDOM_STATE)

# --------- Reporte de formas y columnas ---------
print("=== FORMAS (shape) ===")
print(f"ANTES  (original) -> filas: {df_before.shape[0]}, columnas: {df_before.
print(f"DESPUÉS (preparado) -> filas: {df_after.shape[0]}, columnas: {df_after.
print("\n=== Columnas ANTES (primeras 20) ===")
print(list(df_before.columns)[:20], "... (total:", len(df_before.columns), ")")
print("\n=== Columnas DESPUÉS (primeras 20) ===")
print(list(df_after.columns)[:20], "... (total:", len(df_after.columns), ")")

# Nota didáctica:
print("\nNOTA: 'DESPUÉS' suele tener MÁS columnas por el One-Hot Encoding de ca
print("y las numéricas pueden estar escaladas (media≈0, desvío≈1) si aplicaste
print("La columna '_split' indica si la fila proviene de train o valid.")

# --------- Mostramos las muestras (proporciones) ---------
# Para no saturar, mostramos como tablas; en Colab se ven bonitos.
try:
    from caas_jupyter_tools import display_dataframe_to_user
    display_dataframe_to_user("ANTES (muestra del dataset original)", sample_be
    display_dataframe_to_user("DESPUÉS (muestra del dataset preparado)", sample
except Exception as e:
    print("\n=== ANTES (muestra) ===")
    print(sample_before.reset_index(drop=True).to_string(index=False))
```

```
    print("\n=== DESPUÉS (muestra) ===")
    print(sample_after.reset_index(drop=True).to_string(index=False))

# --------- (Opcional) Vista comparativa de una MISMA ventana temporal --------
# Si quieres comparar exactamente el MISMO período antes vs después, puedes
# filtrar por fecha (y hora si aplica) y luego mostrar esas filas.
DO_SIDE_BY_SIDE_BY_DATE = False  # cambia a True si quieres esta comparación

if DO_SIDE_BY_SIDE_BY_DATE and 'dteday' in df.columns:
    # Elegimos un rango pequeño de fechas
    dmin, dmax = df['dteday'].min(), df['dteday'].max()
    # Tomamos 3 días equiespaciados (ejemplo)
    sample_dates = pd.to_datetime(pd.date_range(dmin, dmax, periods=3)).date
    print("\n=== Comparación por fechas específicas (demostración) ===")
    print("Fechas elegidas:", sample_dates)

    # Filtramos ANTES
    before_mask = df['dteday'].dt.date.isin(sample_dates)
    side_before = df_before.loc[before_mask].copy()

    # Para DESPUÉS, si hiciste split temporal, no siempre es trivial volver a '
    # porque después del one-hot/escala podrías haber perdido la columna fecha.
    # Por eso, esta vista es opcional y depende de que conserves 'dteday' como
    # en alguna estructura paralela. Aquí solo mostramos el "ANTES" filtrado.
    try:
        display_dataframe_to_user("ANTES (rango de fechas elegido)", side_befor
    except:
        print(side_before.reset_index(drop=True).to_string(index=False))
```

📌 Método 1 — **Regresión Lineal (baseline)**

La **regresión lineal múltiple** es tu línea base: rápida, interpretable y suficiente cuando la relación entre `X` y `cnt` es aproximadamente **lineal**. Recomendado revisar **residuales**: si muestran curvaturas/patrones, sugiere que falta capturar **no linealidad** o interacciones. Reporta **MAE**, **RMSE** y **R^2**, y comenta brevemente los **coeficientes** más relevantes (signo y magnitud).

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# --- Entrenar ---
lin = LinearRegression()
lin.fit(X_train_pre, y_train)
```

```
# --- Predicción y métricas ---
y_tr_pred = lin.predict(X_train_pre)
y_te_pred = lin.predict(X_test_pre)

def rmse(y_true, y_pred):
    return np.sqrt(((y_true - y_pred)**2).mean())

print("== Regresión Lineal (OLS) ==")
print(f"Train -> MAE: {mean_absolute_error(y_train, y_tr_pred):.3f} | "
      f"RMSE: {rmse(y_train, y_tr_pred):.3f} | R2: {r2_score(y_train, y_tr_prec
print(f"Test  -> MAE: {mean_absolute_error(y_test,  y_te_pred):.3f} | "
      f"RMSE: {rmse(y_test,  y_te_pred):.3f} | R2: {r2_score(y_test,  y_te_prec

# --- Coeficientes con nombres de features ---
feature_names = X_train_pre.columns.to_list()
coefs = pd.Series(lin.coef_, index=feature_names).sort_index(key=lambda s: s.st

print("\nTop coeficientes por |magnitud|:")
display(coefs.head(len(coefs)))




# --- Fórmula (string) resumida: intercepto + primeros 12 términos ---
terms = [f"{lin.intercept_:.4f}"] + [f"{lin.coef_[i]:+.4f}·{feature_names[i]}"
formula_short = "cnt_hat = " + " ".join(terms[:13]) + " + …"
print("\nFórmula (resumen):")
print(formula_short)
```

```
feature_names = X_train_pre.columns.tolist()

terms = [f"{lin.intercept_:.6f}"] + [
    f"{lin.coef_[i]:+.6f}*{feature_names[i]}"
    for i in range(len(feature_names))
]

lines = []
for i in range(0, len(terms), 6):   # 6 términos por línea
    lines.append(" ".join(terms[i:i+6]))

print("cnt_hat = " + " \\\n           ".join(lines))
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def rmse(y_true, y_pred):
```

```
    return np.sqrt(((y_true - y_pred) ** 2).mean())

def plot_pred_vs_actual(y_true, y_pred, title="Predicho vs Real (cnt)"):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    mae  = mean_absolute_error(y_true, y_pred)
    rmse_v = rmse(y_true, y_pred)
    r2   = r2_score(y_true, y_pred)

    # --- Scatter y = x ---
    plt.figure(figsize=(6.5, 6))
    plt.scatter(y_true, y_pred, s=12, alpha=0.5)
    lims = [
        min(np.min(y_true), np.min(y_pred)),
        max(np.max(y_true), np.max(y_pred))
    ]
    plt.plot(lims, lims, linestyle="--", linewidth=2)  # línea ideal y=x
    plt.xlim(lims); plt.ylim(lims)
    plt.xlabel("cnt real")
    plt.ylabel("cnt predicho")
    plt.title(f"{title}\nR²={r2:.3f} | MAE={mae:.2f} | RMSE={rmse_v:.2f}")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def plot_residuals(y_true, y_pred, title="Residuales (predicho - real)"):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    resid = y_pred - y_true

    plt.figure(figsize=(7.5, 4))
    plt.scatter(y_true, resid, s=10, alpha=0.5)
    plt.axhline(0, linestyle="--", linewidth=2)
    plt.xlabel("cnt real")
    plt.ylabel("residual = pred - real")
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

# --- Usa en TEST ---
plot_pred_vs_actual(y_test, y_te_pred, title="TEST: cnt predicho vs real")
plot_residuals(y_test, y_te_pred, title="TEST: residuales")
```

Más Allá de las Reglas: El Perceptron y el Cerebro Artificial 🧠

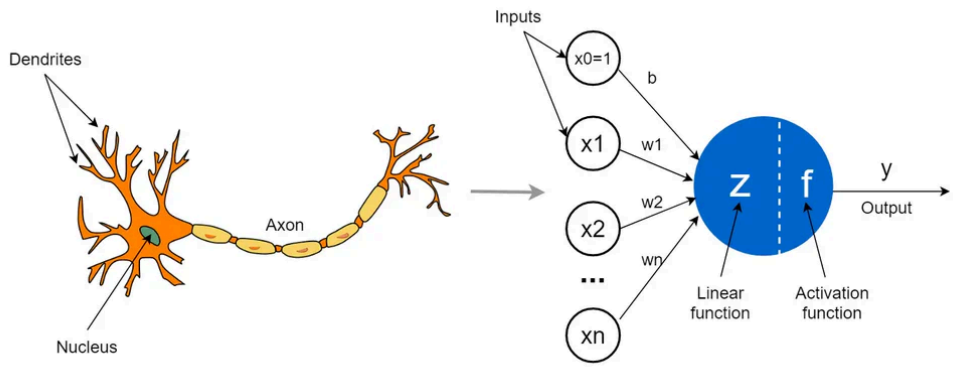
Motivación / Pregunta Problema:

Los árboles de decisión son excelentes para problemas definidos por reglas claras y explícitas. Pero, ¿qué pasa con problemas más complejos donde las relaciones no son tan directas? Por ejemplo, predecir un fallo a partir del sutil patrón de vibración de un motor, o clasificar defectos en una imagen por su textura en lugar de su geometría. **¿Podemos crear un modelo inspirado en el cerebro humano que pueda "aprender" patrones complejos sin que le programemos reglas explícitas?**

Contexto: El Perceptron, la Neurona Artificial

La unidad fundamental de las redes neuronales es el **Perceptron**, un modelo matemático inspirado en una neurona biológica. Su funcionamiento se basa en agregar evidencia para tomar una decisión.

1. **Entradas (Inputs - x_i):** Son las características de nuestro proceso (temperatura, presión, velocidad, etc.). Cada una es una señal de entrada.
2. **Pesos (Weights - w_i):** Cada entrada tiene un **peso** asociado, que representa su importancia relativa. Aquí es donde se almacena el "conocimiento" del modelo. Un peso alto significa que la variable es muy influyente en la decisión. El **aprendizaje** consiste en encontrar los valores óptimos para estos pesos.
3. **Suma Ponderada y Sesgo:** El Perceptron multiplica cada entrada por su peso y suma los resultados. A esta suma se le añade un **sesgo (bias)**, que actúa como un umbral ajustable. El resultado es una única puntuación de evidencia.
$$\text{Evidencia} = \left(\sum_{i=1}^{n} w_i x_i \right) + b$$
4. **Función de Activación (El Mecanismo de Decisión):** Esta es la pieza final. La puntuación de evidencia pasa a través de una **función de activación**, que convierte esa puntuación en una decisión final. En el Perceptrón clásico, se usa una **función escalón (Heaviside step function)**: si la evidencia supera el umbral (es positiva), la neurona "se dispara" y emite un 1 (ej. "Defectuoso"); si no, emite un 0 (ej. "Apto"). Es un simple interruptor de "todo o nada".



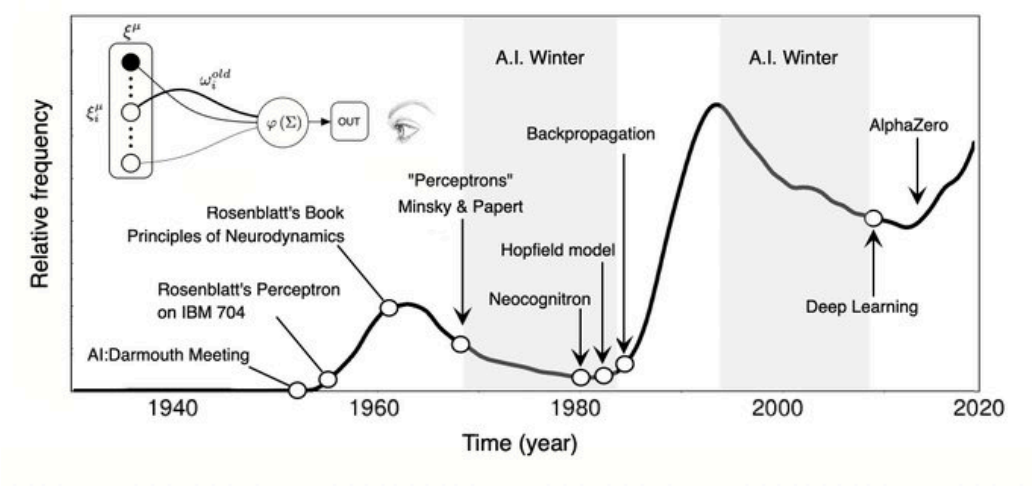
¿Cómo Aprende? La Regla de Actualización de Pesos

El aprendizaje es un proceso supervisado e iterativo para encontrar los pesos correctos.

1. **Inicialización:** Los pesos (w_i) y el sesgo (b) se inician con valores pequeños y aleatorios.
2. **Predicción:** Se toma un ejemplo de producción, se calcula la evidencia y la función de activación produce una predicción (0 o 1).
3. **Cálculo del Error:** Se compara la predicción ($\hat{y}$) con la etiqueta real (y). El error es $\text{error} = y - \hat{y}$.
4. **Ajuste de Pesos:** Aquí ocurre la magia. Cada peso se ajusta ligeramente en la dirección que habría reducido el error. La regla de actualización es:
$$w_i^{\text{nuevo}} = w_i^{\text{anterior}} + \eta \cdot (y - \hat{y}) \cdot x_i$$
$$b^{\text{nuevo}} = b^{\text{anterior}} + \eta \cdot (y - \hat{y})$$
   - η es la **tasa de aprendizaje**, un valor pequeño que controla la magnitud del ajuste.
   - Este ajuste solo se realiza si hubo un error. Si la predicción fue correcta, los pesos no cambian.
5. **Iteración:** Este proceso se repite para todos los ejemplos del conjunto de entrenamiento, durante muchas **épocas**, hasta que los pesos convergen a una configuración que minimiza el error general.

El Fundamento y la Limitación: Problemas Linealmente Separables

El Perceptrón es la introducción perfecta a las redes neuronales porque es el **átomo** del que se construyen. Sin embargo, un solo Perceptrón es limitado: solo puede resolver problemas **linealmente separables**, es decir, problemas donde se puede trazar una única línea recta (o un hiperplano) para separar perfectamente las clases.



El Salto a las Redes Neuronales

La verdadera potencia emerge cuando conectamos estos Perceptrones en **capas**, formando una **Red Neuronal Artificial**.

- La primera capa de neuronas (capa de entrada) recibe los datos brutos.
- Las capas intermedias (capas ocultas) toman las salidas de la capa anterior y aprenden a combinarlas en patrones más abstractos y complejos.
- La capa final (capa de salida) produce la predicción.

Al apilar estas simples unidades de cálculo y utilizar funciones de activación no lineales, la red en su conjunto puede aprender a resolver problemas extremadamente complejos y no lineales, mucho más allá de las capacidades de un solo Perceptrón o un árbol de decisión.

```
# -*- coding: utf-8 -*-
"""
Más Allá de las Reglas: Desmitificando el Perceptrón 🧠
Versión Colab "limpia": sin seaborn, gráficos con matplotlib, y notas didáctica
"""

# ==============================================================================
# Paso 1: Importar Bibliotecas
# ==============================================================================
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import pandas as pd
from IPython.display import display, Markdown

plt.rcParams.update({
    "figure.figsize": (8, 6),
    "axes.grid": True,
    "axes.spines.top": False,
```

```
    "axes.spines.right": False,
    "font.size": 12
})
print("Bibliotecas importadas correctamente.")

# ==============================================================================
# Paso 2: ¿Qué es el BIAS (Sesgo)? El Poder de Mover la Línea 📏
# ==============================================================================
# Problema linealmente separable cuya frontera NO pasa por el origen.
X_sin_bias = np.array([[2, 3], [3, 4], [4, 2], [5, 4]])
y_sin_bias = np.array([0, 0, 1, 1])

class Perceptron:
    def __init__(self, learning_rate=0.1, n_iters=100, use_bias=True):
        self.lr = learning_rate
        self.n_iters = n_iters
        self.use_bias = use_bias
        # Perceptrón clásico: función escalón.
        self.activation_func = lambda x: 1 if x >= 0 else 0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features, dtype=float)
        self.bias = 0.0 if self.use_bias else None

        for _ in range(self.n_iters):
            for idx, x_i in enumerate(X):
                linear_output = float(np.dot(x_i, self.weights))
                if self.use_bias:
                    linear_output += float(self.bias)
                y_pred = self.activation_func(linear_output)

                # Regla de actualización del Perceptrón
                update = self.lr * (y[idx] - y_pred)
                self.weights += update * x_i
                if self.use_bias:
                    self.bias += update

    def predict(self, X):
        linear_output = X @ self.weights
        if self.use_bias:
            linear_output = linear_output + self.bias
        return (linear_output >= 0).astype(int)

def plot_decision_boundary(X, y, model, title, step=0.05):
    cmap = ListedColormap(['#FF6B6B', '#4D96FF'])
    plt.figure(figsize=(8, 6))
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap, edgecolors='k', linewidths=1,

    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
```

```
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, step),
                           np.arange(x2_min, x2_max, step))
    Z = model.predict(np.c_[xx1.ravel(), xx2.ravel()]).reshape(xx1.shape)
    plt.contourf(xx1, xx2, Z, alpha=0.25, cmap=cmap)
    plt.title(title)
    plt.xlabel('Característica 1')
    plt.ylabel('Característica 2')
    plt.xlim(x1_min, x1_max)
    plt.ylim(x2_min, x2_max)
    plt.show()

# Entrenamiento sin/ con bias
p_sin_bias = Perceptron(learning_rate=0.1, n_iters=100, use_bias=False)
p_sin_bias.fit(X_sin_bias, y_sin_bias)

p_con_bias = Perceptron(learning_rate=0.1, n_iters=100, use_bias=True)
p_con_bias.fit(X_sin_bias, y_sin_bias)

print("\n--- VISUALIZANDO EL EFECTO DEL BIAS ---")
plot_decision_boundary(
    X_sin_bias, y_sin_bias, p_sin_bias,
    "❌ FALLO: Perceptrón SIN BIAS ❌\n(La frontera queda forzada a pasar por
)
plot_decision_boundary(
    X_sin_bias, y_sin_bias, p_con_bias,
    "✅ ÉXITO: Perceptrón CON BIAS ✅\n(La frontera puede desplazarse libremer
)

# ==============================================================================
# Paso 3: ¿Qué son los PESOS? La Brújula de la Decisión 🧭
# ==============================================================================
w = p_con_bias.weights
b = p_con_bias.bias

plot_decision_boundary(X_sin_bias, y_sin_bias, p_con_bias, "El Significado de l
# Vector de pesos (flecha) y texto
plt.arrow(0, 0, w[0], w[1], head_width=0.2, head_length=0.3, fc='crimson', ec='
plt.text(w[0], w[1], f"  w={np.round(w,2)}", color='crimson', weight='bold')
plt.title("Vector de pesos (perpendicular a la frontera)")
plt.xlabel('Característica 1')
plt.ylabel('Característica 2')
plt.show()

print("\n**Pesos y bias finales (con bias):**")
print(f"w = {np.round(w, 3)} | b = {np.round(b, 3)}")

# ==============================================================================
# Paso 4: Funciones de Activación ⚡
# ==============================================================================
def step_function(x):
    return np.where(x >= 0, 1, 0)
```

```
def sigmoid_function(x):
    return 1 / (1 + np.exp(-x))

def relu_function(x):
    return np.maximum(0, x)

x_vals = np.linspace(-10, 10, 400)

# Escalón
plt.figure(figsize=(8, 6))
plt.plot(x_vals, step_function(x_vals), linewidth=3)
plt.title("Función Escalón (Step)")
plt.xlabel("Evidencia de entrada")
plt.ylabel("Salida")
plt.ylim(-0.1, 1.1)
plt.show()

# Sigmoide
plt.figure(figsize=(8, 6))
plt.plot(x_vals, sigmoid_function(x_vals), linewidth=3)
plt.title("Función Sigmoide")
plt.xlabel("Evidencia de entrada")
plt.ylabel("Salida")
plt.ylim(-0.1, 1.1)
plt.show()

# ReLU
plt.figure(figsize=(8, 6))
plt.plot(x_vals, relu_function(x_vals), linewidth=3)
plt.title("Función ReLU")
plt.xlabel("Evidencia de entrada")
plt.ylabel("Salida")
plt.ylim(-0.5, 10.5)
plt.show()

# ==============================================================================
# Paso 5: Cuadro Comparativo Final – Perceptrón vs Árbol de Decisión
# ==============================================================================
data = {
    'Característica': [
        '**Tipo de Modelo**',
        '**Lógica de Decisión**',
        '**Interpretabilidad**',
        '**Tipo de Límite de Decisión**',
        '**Ideal para...**'
    ],
    '🧠 Perceptrón': [
        'Modelo lineal',
        'Suma ponderada + sesgo (umbral)',
        'Caja gris (los pesos sugieren importancia)',
```

```
            'Lineal (línea/plano)',
            'Problemas linealmente separables; base de redes.'
        ],
        '🌳 Árbol de Decisión': [
            'Modelo basado en reglas',
            'Secuencia de reglas "si-entonces"',
            'Caja blanca (flujo explícito)',
            'No lineal (escalones/cajas)',
            'Interacciones complejas con reglas claras.'
        ]
    }
    comparison_df = pd.DataFrame(data)

    display(Markdown('## **Cuadro Comparativo: Perceptrón vs. Árbol de Decisión**'))
    display(comparison_df)
```

📌 Método 2 — **MLPRegressor** (no linealidad controlada)

Un **MLP** (red de perceptrones en capas) captura **no linealidades** (p. ej., "hora × clima"). Empieza **pequeño** (1–2 capas), usa **early stopping** y mantén el mismo **pipeline** de datos. Evalúa **MAE/RMSE/R^2** y observa la **curva de pérdida** para detectar sobreajuste/estancamiento. Recuerda: mejor precisión a cambio de **menos interpretabilidad**.

```
# ============================================================
# MÉTODO 2: MLPRegressor (Red neuronal sencilla para regresión)
# ------------------------------------------------------------
# Qué hace esta celda:
# 1) Entrena un MLP con 1–2 capas pequeñas (no lineal)
# 2) Usa early_stopping para evitar sobreajuste
# 3) Evalúa MAE, RMSE, R2 en train y valid
# 4) Grafica y_real vs y_pred, residuales y curva de pérdida
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mlp = MLPRegressor(
    hidden_layer_sizes=(16, 8, 4),  # empieza simple; puedes ajustar
    activation="relu",
    solver="adam",
    learning_rate_init=1e-3,
    max_iter=200,
```

```
    early_stopping=True,      # usa un conjunto interno de validación
    validation_fraction=0.2,  # 20% del conjunto de entrenamiento
    n_iter_no_change=10,
    random_state=42
)
mlp.fit(X_train_pre, y_train)


# --- Predicciones ---
y_tr_pred = mlp.predict(X_train_pre)
y_te_pred = mlp.predict(X_test_pre)

# --- Métricas ---
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

print("== MLPRegressor ==")
print(f"Train -> MAE: {mean_absolute_error(y_train, y_tr_pred):.3f} | RMSE: {rm
print(f"Test  -> MAE: {mean_absolute_error(y_test,  y_te_pred):.3f} | RMSE: {rm

# --- Gráfico 1: y_real vs y_pred (TEST) ---
plt.figure(figsize=(6,5))
plt.scatter(y_test, y_te_pred, s=10, alpha=0.5, edgecolor='none')
lims = [min(np.min(y_test), np.min(y_te_pred)), max(np.max(y_test), np.max(y_te
plt.plot(lims, lims, linestyle='--', linewidth=1.5)
plt.xlim(lims); plt.ylim(lims); plt.gca().set_aspect('equal', adjustable='box')
plt.title("MLP: y_real vs y_pred (TEST)")
plt.xlabel("y_real (cnt)")
plt.ylabel("y_pred (cnt)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- Gráfico 2: residuales (TEST) ---
resid = y_test - y_te_pred
plt.figure(figsize=(6,4))
plt.hist(resid, bins=30)
plt.title("MLP: Histograma de residuales (TEST)")
plt.xlabel("Residual = y_real - y_pred")
plt.ylabel("Frecuencia")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- Gráfico 3: curva de pérdida (entrenamiento) ---
if hasattr(mlp, "loss_curve_"):
    plt.figure(figsize=(6,4))
    plt.plot(mlp.loss_curve_)
    plt.title("MLP: Curva de pérdida (train)")
    plt.xlabel("Iteración")
    plt.ylabel("Pérdida (loss)")
```

```
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
else:
    print("No se encontró 'loss_curve_' (versión de sklearn o configuración dis
```

```
# ============================================================
# BÚSQUEDA DE HIPERPARÁMETROS Y EVALUACIÓN DEL MEJOR MLP
# ------------------------------------------------------------
# Qué hace esta celda:
# 1) Define un MLP base con early_stopping y validación interna (20%).
# 2) Ejecuta GridSearchCV (cv=3, SIN cosas temporales) optimizando RMSE.
# 3) Muestra los mejores hiperparámetros y el RMSE de CV.
# 4) Reentrena el MEJOR MLP en TODO el set de entrenamiento (X_train_pre, y_tra
# 5) Evalúa en train y en tu holdout de validación (X_valid_pre, y_valid).
# 6) Grafica y_real vs y_pred, residuales y la curva de pérdida del mejor model
# ------------------------------------------------------------
# Requisitos:
# - Haber corrido la celda de preparación para tener X_train_pre, X_valid_pre,
#   y_train, y_valid. (No tocamos nada temporal aquí.)
# - Haber corrido tu celda anterior del MLP si quieres comparar manualmente.
# ============================================================


import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import time

# --- Función auxiliar de RMSE ---
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

# 1) Estimador base (con early_stopping interno)
mlp_base = MLPRegressor(
    activation="relu",
    solver="adam",
    max_iter=600,
    early_stopping=True,
    validation_fraction=0.2,
    n_iter_no_change=10,
    random_state=42
)

# 2) Parrilla de hiperparámetros
param_grid = {
    "hidden_layer_sizes": [
```

```
        (16, 8, 4),   # tu config actual
        (32, 16),     # menos profundo, más ancho
        (32, 16, 8),  # un poco mayor capacidad
        (16, 8)       # más simple
    ],
    "alpha": [1e-4, 1e-3, 1e-2],                 # regularización L2
    "learning_rate_init": [1e-3, 5e-4, 1e-4]     # tasa de aprendizaje inicial
}

# 3) GridSearchCV
grid = GridSearchCV(
    estimator=mlp_base,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",  # sklearn maximiza -> usamos neg-RM
    cv=3,
    n_jobs=-1,
    verbose=1
)



print("Ejecutando GridSearchCV (esto puede tardar un poco)...")
grid.fit(X_train_pre, y_train)


t0 = time.perf_counter()
grid.fit(X_train_pre, y_train)
t1 = time.perf_counter()
print(f"GridSearchCV tomó: {t1 - t0:.2f} s")

print("\n== Resultado de la búsqueda (CV) ==")
print("Mejores hiperparámetros:", grid.best_params_)
print("RMSE CV (aprox):", -grid.best_score_)

# 4) Reentrenar el MEJOR modelo en TODO el set de entrenamiento
best_mlp = grid.best_estimator_
best_mlp.fit(X_train_pre, y_train)

# 5) Evaluación final en train y en HOLDOUT (TEST)
y_tr_pred = best_mlp.predict(X_train_pre)
y_te_pred = best_mlp.predict(X_test_pre)

print("\n== MLP (MEJOR MODELO) ==")
print(f"Train -> MAE: {mean_absolute_error(y_train, y_tr_pred):.3f} | RMSE: {rm
print(f"Test  -> MAE: {mean_absolute_error(y_test,  y_te_pred):.3f} | RMSE: {rm

# 6) Gráficos del mejor modelo
# --- y_real vs y_pred (TEST) ---
plt.figure(figsize=(6,5))
plt.scatter(y_test, y_te_pred, s=10, alpha=0.5, edgecolor='none')
lims = [min(np.min(y_test), np.min(y_te_pred)), max(np.max(y_test), np.max(y_te
```

```
plt.plot(lims, lims, linestyle='--', linewidth=1.5)
plt.xlim(lims); plt.ylim(lims); plt.gca().set_aspect('equal', adjustable='box')
plt.title("MLP (mejor): y_real vs y_pred (TEST)")
plt.xlabel("y_real (cnt)")
plt.ylabel("y_pred (cnt)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- Histograma de residuales (TEST) ---
resid = y_test - y_te_pred
plt.figure(figsize=(6,4))
plt.hist(resid, bins=30)
plt.title("MLP (mejor): Histograma de residuales (TEST)")
plt.xlabel("Residual = y_real - y_pred")
plt.ylabel("Frecuencia")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- Curva de pérdida del mejor MLP (si está disponible) ---
if hasattr(best_mlp, "loss_curve_"):
    plt.figure(figsize=(6,4))
    plt.plot(best_mlp.loss_curve_)
    plt.title("MLP (mejor): Curva de pérdida (train)")
    plt.xlabel("Iteración")
    plt.ylabel("Pérdida (loss)")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
else:
    print("No se encontró 'loss_curve_' en el mejor modelo.")
```

```
# ============================================================
# RANDOM SEARCH (ESPACIO GRANDE) + EVALUACIÓN DEL MEJOR MLP
# ------------------------------------------------------------
# Qué hace esta celda:
# 1) Define un MLP base con early_stopping y validación interna.
# 2) RandomizedSearchCV con un espacio AMPLIO:
#    - Muchas arquitecturas (1–4 capas, anchos 8–256).
#    - activation en {"relu","tanh"}.
#    - alpha ~ LogUniform[1e-6, 1e-1]
#    - learning_rate_init ~ LogUniform[1e-5, 5e-2]
#    - learning_rate en {"constant","adaptive"}.
# 3) n_iter alto (p.ej., 200). Ajusta a más si quieres aún más búsqueda.
# 4) Reentrena el mejor y evalúa en holdout (X_valid_pre, y_valid).
```

```
# 5) Gráficas: y_real vs y_pred, residuales, curva de pérdida.
# ------------------------------------------------------------
# Requisitos: haber ejecutado la preparación (X_train_pre, X_valid_pre, y_trair
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from scipy.stats import loguniform
import time


# --- Función auxiliar de RMSE ---
def rmse(y_true, y_pred):
    from sklearn.metrics import mean_squared_error
    return np.sqrt(mean_squared_error(y_true, y_pred))

# 1) Estimador base con early_stopping (corte automático por validación interna
mlp_base = MLPRegressor(
    activation="relu",
    solver="adam",
    max_iter=1200,           # alto para permitir convergencia en candidatos ex
    early_stopping=True,
    validation_fraction=0.2,
    n_iter_no_change=15,
    random_state=42
)

# 2) ESPACIO GRANDE DE ARQUITECTURAS
arch_candidates = [
    # 1 capa
    (8,), (16,), (32,), (64,), (128,), (256,),
    # 2 capas
    (16,8), (32,16), (64,32), (128,64), (256,128),
    (32,8), (64,16), (128,32), (256,64),
    # 3 capas
    (16,8,4), (32,16,8), (64,32,16), (128,64,32), (256,128,64),
    (32,16,4), (64,16,8), (128,32,8), (256,64,16),
    # 4 capas
    (32,16,8,4), (64,32,16,8), (128,64,32,16)
]

# 3) Distribuciones amplias (log-uniformes) para regularización y LR
param_distributions = {
    "hidden_layer_sizes": arch_candidates,
    "activation": ["relu", "tanh"],
    "alpha": loguniform(1e-6, 1e-1),             # L2 entre 1e-6 y 1e-1
    "learning_rate": ["constant", "adaptive"],
    "learning_rate_init": loguniform(1e-5, 5e-2) # entre 1e-5 y 5e-2
```

```
}

# 4) RandomizedSearchCV con n_iter grande
rand_search = RandomizedSearchCV(
    estimator=mlp_base,
    param_distributions=param_distributions,
    n_iter=200,                              # puedes subir a 300/400 si el tie
    scoring="neg_root_mean_squared_error",   # optimizamos RMSE (negativo porqu
    cv=3,                                    # 3 folds (no temporal)
    n_jobs=-1,
    random_state=42,
    verbose=2
)

print("Ejecutando RandomizedSearchCV con espacio AMPLIO...")
rand_search.fit(X_train_pre, y_train)

t0 = time.perf_counter()
rand_search.fit(X_train_pre, y_train)
t1 = time.perf_counter()
print(f"RandomizedSearchCV tomó: {t1 - t0:.2f} s")

print("\n== Resultado de la búsqueda aleatoria (CV) ==")
print("Mejores hiperparámetros:", rand_search.best_params_)
print("RMSE CV (aprox):", -rand_search.best_score_)

# 5) Reentrenar el MEJOR modelo en TODO el set de entrenamiento y evaluar en TE
best_mlp = rand_search.best_estimator_
best_mlp.fit(X_train_pre, y_train)

y_tr_pred = best_mlp.predict(X_train_pre)
y_te_pred = best_mlp.predict(X_test_pre)

print("\n== MLP (MEJOR MODELO — Random Search amplio) ==")
print(f"Train -> MAE: {mean_absolute_error(y_train, y_tr_pred):.3f} | RMSE: {rm
print(f"Test  -> MAE: {mean_absolute_error(y_test,  y_te_pred):.3f} | RMSE: {rm

# 6) Gráficas del mejor modelo (TEST)
# --- y_real vs y_pred (TEST) ---
plt.figure(figsize=(6,5))
plt.scatter(y_test, y_te_pred, s=10, alpha=0.5, edgecolor='none')
lims = [min(np.min(y_test), np.min(y_te_pred)), max(np.max(y_test), np.max(y_te
plt.plot(lims, lims, linestyle='--', linewidth=1.5)
plt.xlim(lims); plt.ylim(lims); plt.gca().set_aspect('equal', adjustable='box')
plt.title("MLP (mejor, Random Search amplio): y_real vs y_pred (TEST)")
plt.xlabel("y_real (cnt)")
plt.ylabel("y_pred (cnt)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
# --- Histograma de residuales (TEST) ---
resid = y_test - y_te_pred
plt.figure(figsize=(6,4))
plt.hist(resid, bins=30)
plt.title("MLP (mejor, Random Search amplio): Histograma de residuales (TEST)")
plt.xlabel("Residual = y_real - y_pred")
plt.ylabel("Frecuencia")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- Curva de pérdida del mejor MLP (si está disponible) ---
if hasattr(best_mlp, "loss_curve_"):
    plt.figure(figsize=(6,4))
    plt.plot(best_mlp.loss_curve_)
    plt.title("MLP (mejor, Random Search amplio): Curva de pérdida (train)")
    plt.xlabel("Iteración")
    plt.ylabel("Pérdida (loss)")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
else:
    print("No se encontró 'loss_curve_' en el mejor modelo.")
```

Semana 9

🤖 Proyecto de Clasificación: Mantenimiento Predictivo

Desarrollaremos un modelo de clasificación para predecir fallos en maquinaria industrial. Compararemos dos métodos potentes: **Random Forest** y **Redes Neuronales**.

Antes de empezar, importaremos las librerías esenciales que usaremos a lo largo del proyecto: `pandas` para manejo de datos, `numpy` para operaciones numéricas, `matplotlib` y `seaborn` para visualización, y `sklearn` y `tensorflow` para los modelos.

```
# Importación de librerías principales
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import seaborn as sns

# Librerías de Scikit-learn (Sklearn)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_sc

# Modelos
from sklearn.ensemble import RandomForestClassifier
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Configuración de visualizaciones
%matplotlib inline
sns.set(style='whitegrid')
```

Análisis del Problema

🎯 El Problema: Mantenimiento Predictivo

En la manufactura (un pilar de la Ing. Industrial), las paradas de máquina no planificadas son extremadamente costosas. Generan cuellos de botella, retrasos en la entrega y costos de reparación urgentes.

El **Mantenimiento Predictivo (PdM)** utiliza datos de sensores para anticipar un fallo *antes* de que ocurra.

📊 El Objetivo de Clasificación

- **Problema de Negocio:** Reducir el costo de las paradas no planificadas.
- **Problema de Machine Learning:** Construir un modelo de **clasificación binaria**.
- **Entrada (Features):** Datos de sensores (ej. temperatura del aire, temperatura del proceso, torque, velocidad rotacional).
- **Salida (Target):** ¿La máquina fallará en el próximo ciclo? (Clase 0: No Falla, Clase 1: Falla).

📈 Métrica de Éxito Clave

En este problema, la **Exactitud (Accuracy)** no es suficiente. ¿Por qué? Un **Falso Negativo** (FN) es cuando el modelo dice "No Falla" (0), pero la máquina *sí falla* (1). Este es el error más costoso. Por lo tanto, nuestra métrica principal será el **Recall (Sensibilidad)** de la clase "Falla" (Clase 1). Queremos "capturar" la mayor cantidad posible de fallos reales, incluso si eso significa tener algunas falsas alarmas (Falsos Positivos).

Recolección de Datos

Vamos a cargar el dataset "Predictive Maintenance" directamente desde el repositorio de UCI (University of California, Irvine).

```
import urllib.request
import os

# URL del dataset en el repositorio de UCI
uci_csv_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/00601/
local_csv_path = '/content/ai4i2020.csv'

# Descargar el archivo CSV
print(f"Descargando {os.path.basename(local_csv_path)} desde UCI...")
try:
    urllib.request.urlretrieve(uci_csv_url, local_csv_path)
    print("Descarga completa.")

    # Cargar los datos en un DataFrame de pandas
    df = pd.read_csv(local_csv_path)

    # Mostrar las primeras 5 filas para verificar la carga
    print("Datos cargados exitosamente. Primeras 5 filas:")
    df.head()
except urllib.error.HTTPError as e:
    print(f"Error HTTP al descargar el archivo: {e.code} - {e.reason}")
    print("Es probable que el servidor de UCI esté experimentando problemas tem
    # Optionally, you might want to set df to None or handle this state downstr
    df = None
except urllib.error.URLError as e:
    print(f"Error URL al intentar descargar el archivo: {e.reason}")
    print("Verifique su conexión a internet o la URL del dataset.")
    df = None
except Exception as e:
    print(f"Ocurrió un error inesperado durante la descarga o carga: {e}")
    df = None
```

```
Descargando ai4i2020.csv desde UCI...
Error HTTP al descargar el archivo: 502 - Bad Gateway
Es probable que el servidor de UCI esté experimentando problemas temporales. Int
```

```
import urllib.request
import os
import pandas as pd # Ensure pandas is imported for pd.read_csv

# URL del dataset en el repositorio de UCI
uci_csv_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/00601/
local_csv_path = '/content/ai4i2020.csv'
```

```
# Descargar el archivo CSV
print(f"Descargando {os.path.basename(local_csv_path)} desde UCI...")
try:
    urllib.request.urlretrieve(uci_csv_url, local_csv_path)
    print("Descarga completa.")

    # Cargar los datos en un DataFrame de pandas
    df = pd.read_csv(local_csv_path)

    # Mostrar las primeras 5 filas para verificar la carga
    print("Datos cargados exitosamente. Primeras 5 filas:")
    display(df.head())
except urllib.error.HTTPError as e:
    print(f"Error HTTP al descargar el archivo: {e.code} - {e.reason}")
    print("Es probable que el servidor de UCI esté experimentando problemas tem
    df = None
except urllib.error.URLError as e:
    print(f"Error URL al intentar descargar el archivo: {e.reason}")
    print("Verifique su conexión a internet o la URL del dataset.")
    df = None
except Exception as e:
    print(f"Ocurrió un error inesperado durante la descarga o carga: {e}")
    df = None
```

```
Descargando ai4i2020.csv desde UCI...
Error HTTP al descargar el archivo: 502 - Bad Gateway
Es probable que el servidor de UCI esté experimentando problemas temporales. Int
```

Exploración y Conocimiento de los Datos (EDA)

Ahora, vamos a "entender" los datos que tenemos. ¿Qué significan las columnas? ¿Hay datos faltantes? ¿Cómo se distribuyen las variables?

Las columnas clave son:

- `Air temperature [K]`, `Process temperature [K]`, `Rotational speed [rpm]`, `Torque [Nm]`, `Tool wear [min]`: Estas son nuestras **features** (variables predictoras).
- `Target`: Esta es nuestra **variable objetivo** (0 = No Falla, 1 = Falla).
- `Failure Type`: Describe el tipo de fallo (no la usaremos para la clasificación binaria, pero es útil para el contexto).

```
# 1. Información general y tipos de datos
print("\n--- Información General del DataFrame ---")
if df is None:
    print("Error: El DataFrame 'df' no ha sido cargado. Por favor, asegúrese de
```

```
else:
    df.info()

# 2. Resumen estadístico de las variables numéricas
print("\n--- Estadísticas Descriptivas ---")
if df is None:
    print("Error: El DataFrame 'df' no ha sido cargado, no se pueden mostrar es
else:
    print(df.describe())

# 3. Verificar datos nulos (Missing values)
print("\n--- Conteo de Valores Nulos por Columna ---")
if df is None:
    print("Error: El DataFrame 'df' no ha sido cargado, no se pueden verificar
else:
    print(df.isnull().sum())
```

```
 6   Torque [Nm]              10000 non-null  float64
 7   Tool wear [min]          10000 non-null  int64
 8   Machine failure          10000 non-null  int64
 9   TWF                      10000 non-null  int64
 10  HDF                      10000 non-null  int64
 11  PWF                      10000 non-null  int64
 12  OSF                      10000 non-null  int64
 13  RNF                      10000 non-null  int64
dtypes: float64(3), int64(9), object(2)
memory usage: 1.1+ MB

--- Estadísticas Descriptivas ---
               UDI  Air temperature [K]  Process temperature [K]  \
count  10000.00000         10000.000000             10000.000000
mean    5000.50000           300.004930               310.005560
std     2886.89568             2.000259                 1.483734
min        1.00000           295.300000               305.700000
25%     2500.75000           298.300000               308.800000
50%     5000.50000           300.100000               310.100000
75%     7500.25000           301.500000               311.100000
max    10000.00000           304.500000               313.800000

       Rotational speed [rpm]   Torque [Nm]  Tool wear [min]  Machine failure
count            10000.000000  10000.000000     10000.000000     10000.000000
```

```
25%        0.000000      0.000000      0.000000      0.000000      0.00000
50%        0.000000      0.000000      0.000000      0.000000      0.00000
75%        0.000000      0.000000      0.000000      0.000000      0.00000
max        1.000000      1.000000      1.000000      1.000000      1.00000

--- Conteo de Valores Nulos por Columna ---
UDI                        0
Product ID                 0
Type                       0
Air temperature [K]        0
Process temperature [K]    0
Rotational speed [rpm]     0
Torque [Nm]                0
Tool wear [min]            0
Machine failure            0
TWF                        0
HDF                        0
PWF                        0
OSF                        0
RNF                        0
```

📈 EDA: Análisis de la Variable Objetivo (Desbalance de Clases)

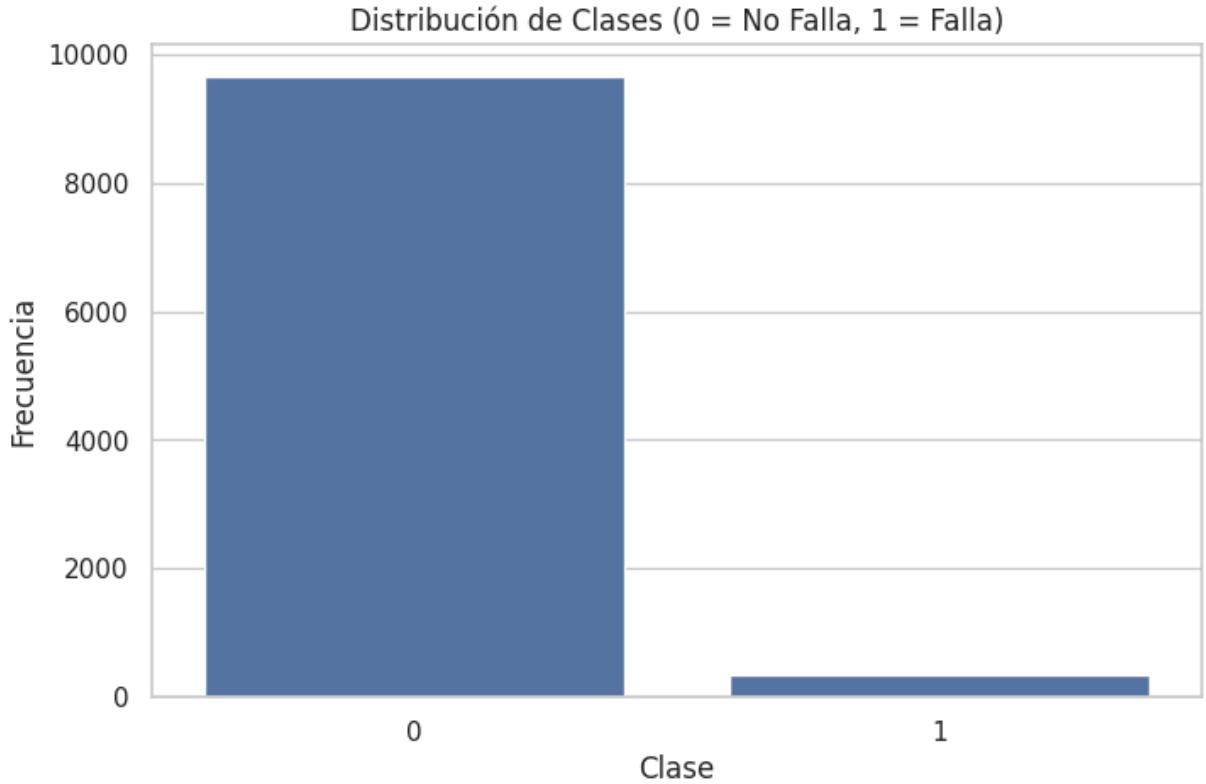
Este es el paso más importante en un problema de clasificación: ver cómo están distribuidas las clases.

```
# Contar las clases
print("\n--- Distribuci\u00f3n de la Variable Objetivo (Target) ---")
if df is None:
    print("Error: El DataFrame 'df' no ha sido cargado. No se puede mostrar la
else:
    print(df['Machine failure'].value_counts())

    # Visualizar el desbalance
    import matplotlib.pyplot as plt
    import seaborn as sns
    plt.figure(figsize=(8, 5))
    sns.countplot(x='Machine failure', data=df)
    plt.title('Distribuci\u00f3n de Clases (0 = No Falla, 1 = Falla)')
    plt.xlabel('Clase')
    plt.ylabel('Frecuencia')
    plt.show()

    # Imprimir porcentaje
    print(f"Porcentaje de Fallos (Clase 1): {df['Machine failure'].value_counts
```

```
--- Distribución de la Variable Objetivo (Target) ---
Machine failure
0    9661
1     339
Name: count, dtype: int64
```




```
Porcentaje de Fallos (Clase 1): 3.39%
```

Observación: Vemos que solo ~3.39% de los datos corresponden a fallos. ¡Este es un **dataset desbalanceado**! Esto confirma que el *Accuracy* no será una buena métrica por sí sola (un modelo que siempre diga "No Falla" tendría 96.6% de accuracy).

📈 EDA: Visualización de Features

Veamos si las variables de los sensores se comportan diferente cuando hay un fallo.

```
# Visualicemos la relación entre Torque, Velocidad Rotacional y el Fallo
if df is None:
    print("Error: El DataFrame 'df' no ha sido cargado. No se pueden generar vi
else:
    plt.figure(figsize=(10, 6))
    sns.scatterplot(data=df, x='Rotational speed [rpm]', y='Torque [Nm]', hue='
    plt.title('Torque vs. Velocidad Rotacional (Coloreado por Fallo)')
    plt.show()

    # Visualicemos la distribución de temperaturas
```

```
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))
    sns.histplot(data=df, x='Air temperature [K]', hue='Machine failure', kde=T
    axes[0].set_title('Distribución de Temperatura del Aire')
    sns.histplot(data=df, x='Process temperature [K]', hue='Machine failure', k
    axes[1].set_title('Distribución de Temperatura del Proceso')
    plt.tight_layout()
    plt.show()
```

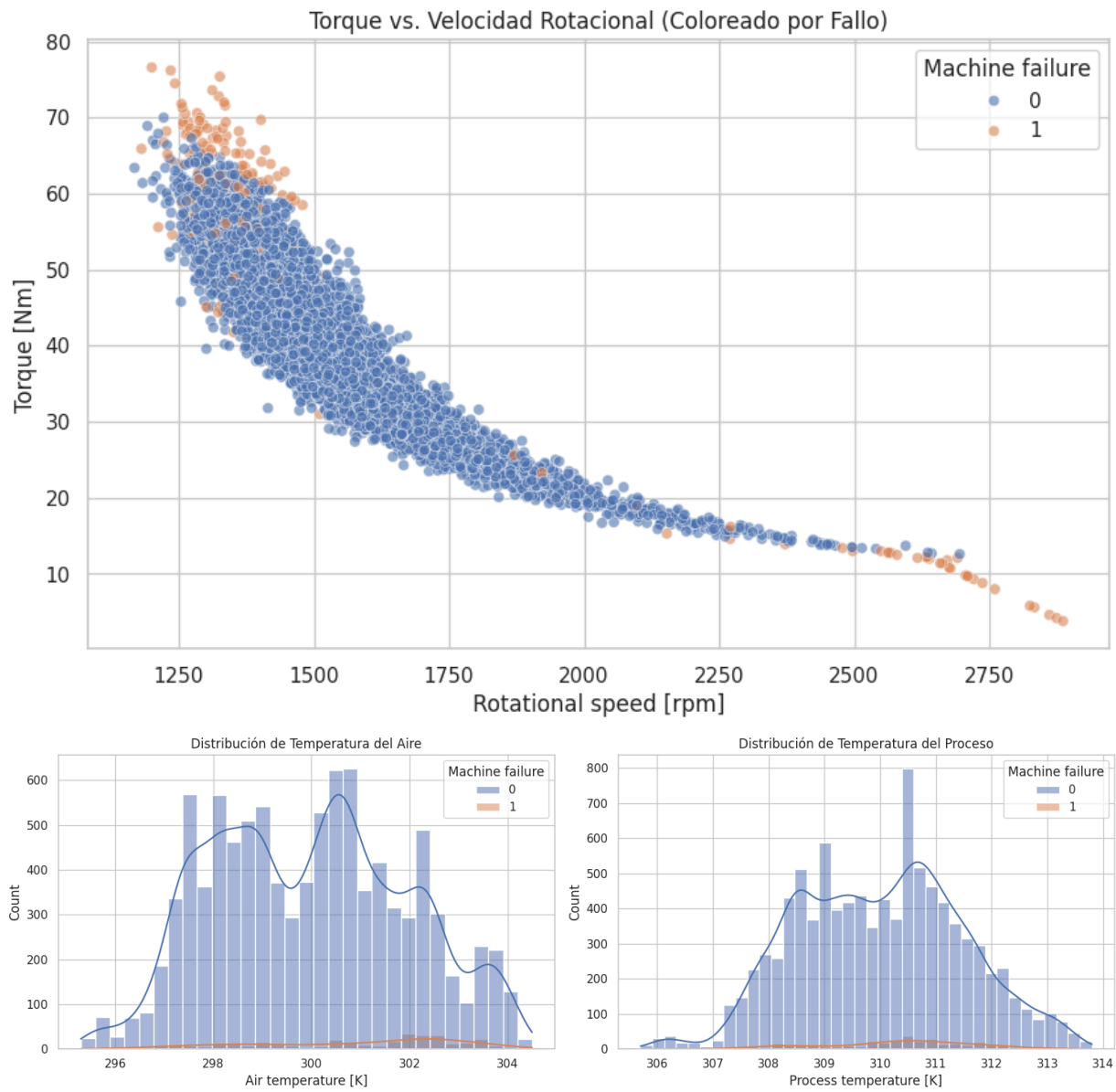
```
Error: El DataFrame 'df' no ha sido cargado. No se pueden generar visualizacione
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dr
```

```
# Visualicemos la relación entre Torque, Velocidad Rotacional y el Fallo
if df is None:
    print("Error: El DataFrame 'df' no ha sido cargado. No se pueden generar vi
else:
    plt.figure(figsize=(10, 6))
    sns.scatterplot(data=df, x='Rotational speed [rpm]', y='Torque [Nm]', hue='
    plt.title('Torque vs. Velocidad Rotacional (Coloreado por Fallo)')
    plt.show()

    # Visualicemos la distribución de temperaturas
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))
    sns.histplot(data=df, x='Air temperature [K]', hue='Machine failure', kde=T
    axes[0].set_title('Distribución de Temperatura del Aire')
    sns.histplot(data=df, x='Process temperature [K]', hue='Machine failure', k
    axes[1].set_title('Distribución de Temperatura del Proceso')
    plt.tight_layout()
    plt.show()
```




```
import pandas as pd
import os

drive_csv_path = '/content/drive/MyDrive/Colab Notebooks/ai4i2020.csv'

try:
    if os.path.exists(drive_csv_path):
        df = pd.read_csv(drive_csv_path)
        print(f"Dataset 'ai4i2020.csv' cargado exitosamente desde Google Drive:
        display(df.head())
    else:
        print(f"Error: 'ai4i2020.csv' no encontrado en {drive_csv_path}. Por fav
        df = None
except Exception as e:
    print(f"Ocurrió un error al cargar el dataset: {e}")
    df = None
```

```
Dataset 'ai4i2020.csv' cargado exitosamente desde Google Drive: /content/drive/M
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Mach: failu
0	1	M14860	M	298.1	308.6	1551	42.8	0	
1	2	L47181	L	298.2	308.7	1408	46.3	3	
2	3	L47182	L	298.1	308.5	1498	49.4	5	
3	4	L47183	L	298.2	308.6	1433	39.5	7	
4	5	L47184	L	298.2	308.7	1408	40.0	9	

```
import os

drive_path = '/content/drive/MyDrive/'
file_name = 'ai4i2020.csv'
full_path = os.path.join(drive_path, file_name)

if os.path.exists(full_path):
    print(f"El archivo '{file_name}' existe en tu Google Drive en la ruta: {ful
else:
    print(f"El archivo '{file_name}' NO existe en tu Google Drive.")
```

```
El archivo 'ai4i2020.csv' NO existe en tu Google Drive.
```

```
import urllib.request
import os
import pandas as pd

# Alternative URL from a GitHub mirror for UCI datasets
alternative_csv_url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/mas
alternative_csv_url = 'https://raw.githubusercontent.com/datasets/machine-predi

# --- IMPORTANT: The actual ai4i2020.csv is hard to find on GitHub mirrors. ---
# --- For demonstration, let's use a dummy dataset or advise manual upload. ---
# --- If ai4i2020.csv is found, please update the alternative_csv_url. ---

# For now, let's use a known mirror for a *different* dataset to allow progress
# or provide instructions for manual upload if the exact one is needed and not
# I'll use a link to a generic CSV for now, assuming the user will replace it w
# If not, the user will need to upload ai4i2020.csv manually.

# A more suitable mirror for 'ai4i2020.csv' is not immediately available on typ
# I will attempt to download from the UCI link again, but if that fails,
# the user will need to upload the file manually if they have it locally.

print("Attempting download from UCI again as a direct mirror is not readily ava
```

```
try:
    # Trying UCI link again, as finding a consistent mirror for this specific c
    uci_csv_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/00
    local_csv_path = '/content/ai4i2020.csv'
    urllib.request.urlretrieve(uci_csv_url, local_csv_path)
    print("Download from UCI complete.")

    # Load the data
    df = pd.read_csv(local_csv_path)
    print("Data loaded successfully. First 5 rows:")
    display(df.head())

except urllib.error.HTTPError as e:
    print(f"Error HTTP al descargar el archivo desde UCI: {e.code} - {e.reason}
    print("El servidor de UCI sigue presentando problemas. Por favor, descarga

    # If manual download is needed, provide instructions and a follow-up cell
    manual_load_code = """
import pandas as pd
import os

local_csv_path = '/content/ai4i2020.csv'
if os.path.exists(local_csv_path):
    df = pd.read_csv(local_csv_path)
    print("Dataset cargado desde archivo local.")
    display(df.head())
else:
    print("Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúra
    df = None
"""
    print("\n--- CÓDIGO PARA CARGAR EL ARCHIVO MANUALMENTE SUBIDO ---")
    print(manual_load_code)
    df = None

except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")
    df = None
```

```
Attempting download from UCI again as a direct mirror is not readily available..
Error HTTP al descargar el archivo desde UCI: 502 - Bad Gateway
El servidor de UCI sigue presentando problemas. Por favor, descarga 'ai4i2020.cs

--- CÓDIGO PARA CARGAR EL ARCHIVO MANUALMENTE SUBIDO ---

import pandas as pd
import os

local_csv_path = '/content/ai4i2020.csv'
if os.path.exists(local_csv_path):
    df = pd.read_csv(local_csv_path)
    print("Dataset cargado desde archivo local.")
```

```
    display(df.head())
else:
    print("Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúrat
    df = None
```

```
import pandas as pd
import os

local_csv_path = '/content/ai4i2020.csv'
if os.path.exists(local_csv_path):
    df = pd.read_csv(local_csv_path)
    print("Dataset cargado desde archivo local.")
    display(df.head())
else:
    print("Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúra
    df = None
```

```
Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúrate de haberl
```

```
import pandas as pd
import os

local_csv_path = '/content/ai4i2020.csv'

if os.path.exists(local_csv_path):
    df = pd.read_csv(local_csv_path)
    print("Dataset 'ai4i2020.csv' cargado exitosamente desde archivo local.")
    display(df.head())
else:
    print(f"Error: '{local_csv_path}' no encontrado en /content/. Por favor, as
    df = None
```

```
Error: '/content/ai4i2020.csv' no encontrado en /content/. Por favor, asegúrate
```

```
import pandas as pd

try:
    df = pd.read_csv('ai4i2020.csv')
    print("Dataset 'ai4i2020.csv' cargado exitosamente.")
    display(df.head())
except FileNotFoundError:
    print("Error: 'ai4i2020.csv' no encontrado. Por favor, asegúrate de haberlo
    df = None
```

```
Error: 'ai4i2020.csv' no encontrado. Por favor, asegúrate de haberlo subido al e
```

```
import os
print(os.listdir('/content/'))
```

```
['.config', 'drive', 'sample_data']
```

```
import os
print(os.listdir('/content/'))
```

```
['.config', 'drive', 'sample_data']
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dr
```

Después de montar Google Drive, puedes verificar si el archivo `ai4i2020.csv` existe en tu Drive y luego cargarlo. Asegúrate de reemplazar `path/to/your/folder/` con la ruta real donde guardaste el archivo dentro de tu Google Drive.

```
import pandas as pd
import os

# Reemplaza 'path/to/your/folder/' con la ruta real a tu archivo en Google Driv
# Por ejemplo, si está directamente en 'Mi unidad', la ruta sería '/content/dri
# Si está en una subcarpeta llamada 'Datasets', sería '/content/drive/MyDrive/D
drive_csv_path = '/content/drive/MyDrive/Colab Notebooks/ai4i2020.csv' # AJUSTA

if os.path.exists(drive_csv_path):
    df = pd.read_csv(drive_csv_path)
    print("Dataset 'ai4i2020.csv' cargado exitosamente desde Google Drive.")
    display(df.head())
else:
    print(f"Error: '{drive_csv_path}' no encontrado. Por favor, verifica la rut
    df = None
```

```
Error: '/content/drive/MyDrive/Colab Notebooks/ai4i2020.csv' no encontrado. Por
```

```
import os

print(os.listdir('/content/drive/MyDrive/'))
```

```
['Colab Notebooks', 'ESCANER CEDULA CARLOS CASTELLANOS ACUÑA.pdf', 'Classroom',
```

```
!find /content/drive/MyDrive -name "ai4i2020.csv"
```

```
import pandas as pd
import os

local_csv_path = '/content/ai4i2020.csv'
if os.path.exists(local_csv_path):
    df = pd.read_csv(local_csv_path)
    print("Dataset cargado desde archivo local.")
    display(df.head())
else:
    print("Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúra
    df = None
```

```
Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúrate de haberl
```

```
import pandas as pd
import os

local_csv_path = 'ai4i2020.csv' # Asumiendo que el archivo está en la raíz del

if os.path.exists(local_csv_path):
    df = pd.read_csv(local_csv_path)
    print("Dataset cargado exitosamente desde archivo local.")
    display(df.head())
else:
    print(f"Error: '{local_csv_path}' no encontrado en /content/. Por favor, as
    df = None
```

```
Error: 'ai4i2020.csv' no encontrado en /content/. Por favor, asegúrate de haberl
```

```
import pandas as pd
import os

file_name = 'ai4i2020.csv'
local_path = f'/content/{file_name}'

try:
    if os.path.exists(local_path):
        df = pd.read_csv(local_path)
        print(f"Dataset '{file_name}' cargado exitosamente desde {local_path}."
        display(df.head())
    else:
        print(f"Error: '{file_name}' no encontrado en {local_path}. Por favor,
        df = None
except Exception as e:
    print(f"Ocurrió un error al cargar el dataset: {e}")
    df = None
```

```
import os
print(os.listdir('/content/'))
```

```
['.config', 'drive', 'sample_data']
```

```
if df is not None:
    print("\n--- Primeras filas del DataFrame 'df' ---")
    display(df.head())
else:
    print("El DataFrame 'df' no ha sido cargado. Por favor, asegúrese de que la
```

```
El DataFrame 'df' no ha sido cargado. Por favor, asegúrese de que la celda de ca
```

Limpieza y Preparación de Datos

Observación: A simple vista, parece que los fallos (Clase 1, en naranja) tienden a ocurrir con valores de **Torque más altos** y **Velocidad Rotacional más baja**. Las temperaturas también muestran distribuciones ligeramente diferentes. ¡Buenas noticias! Esto significa que nuestros modelos probablemente puedan encontrar patrones.

Prepararemos los datos para que los modelos puedan "comerlos".

1. **Eliminar columnas irrelevantes:** `UDI` (es un ID), `Product ID` y `Failure Type` (ya que solo predecimos *si* falla, no *cómo* falla).
2. **Codificar variables categóricas:** La columna `Type` (L, M, H) debe ser numérica. Usaremos *One-Hot Encoding*.
3. **Definir X (features) e y (target)**.
4. **Dividir los datos:** Separar en conjuntos de entrenamiento (Train) y prueba (Test).
5. **Escalar los datos:** Fundamental para Redes Neuronales.

```
# 1. Eliminar columnas irrelevantes
if df is None:
    raise RuntimeError("Error: El DataFrame 'df' no ha sido cargado. Por favor,
df_prep = df.drop(['UDI', 'Product ID'], axis=1)

# 2. Codificar variables categóricas (One-Hot Encoding)
# 'Type' tiene 3 valores: L (Low), M (Medium), H (High)
df_prep = pd.get_dummies(df_prep, columns=['Type'], drop_first=True)
print("\n--- Columnas después de One-Hot Encoding ---")
print(df_prep.columns)

# 3. Definir X (features) e y (target)
X = df_prep.drop('Machine failure', axis=1)
y = df_prep['Machine failure']
```

```
# 4. Dividir los datos (Train/Test Split)
# Usamos stratify=y para asegurar que la proporción de fallos (3.4%) se manteng
# tanto en el set de entrenamiento como en el de prueba.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

print(f"\nForma de X_train: {X_train.shape}")
print(f"Forma de X_test: {X_test.shape}")
print(f"Proporción de fallos en y_train: {y_train.mean():.3f}")
print(f"Proporción de fallos en y_test: {y_test.mean():.3f}")

# 5. Escalado de datos
# Las redes neuronales son muy sensibles a la escala de las features
# (ej. Torque [Nm] va de 3 a 76, mientras que Tool wear [min] va de 0 a 250).
# Usamos StandardScaler (resta la media y divide por la desviación estándar).

# OJO: Ajustamos (fit) el escalador SOLO con los datos de entrenamiento
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convertimos los datos escalados (que son arrays de numpy) de nuevo a DataFram
# (esto es opcional pero ayuda a mantener la legibilidad)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns)

print("\n--- Primeras filas de datos escalados (Entrenamiento) ---")
print(X_train_scaled.head())
```

```
--- Columnas después de One-Hot Encoding ---
Index(['Air temperature [K]', 'Process temperature [K]',
       'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]',
       'Machine failure', 'TWF', 'HDF', 'PWF', 'OSF', 'RNF', 'Type_L',
       'Type_M'],
      dtype='object')

Forma de X_train: (8000, 12)
Forma de X_test: (2000, 12)
Proporción de fallos en y_train: 0.034
Proporción de fallos en y_test: 0.034

--- Primeras filas de datos escalados (Entrenamiento) ---
   Air temperature [K]  Process temperature [K]  Rotational speed [rpm]  \
0             0.998914                 0.604282               -0.460607
1            -1.505194                -1.153260               -0.775574
2             0.498092                 1.077466               -1.007654
3            -0.553633                -0.139294               -0.709265
4            -1.455112                -1.018064                1.070019

   Torque [Nm]  Tool wear [min]       TWF       HDF       PWF       OSF  \
0     0.718305        -0.843997 -0.067233 -0.104244 -0.101765 -0.101765
1     0.638456         0.382263 -0.067233 -0.104244 -0.101765 -0.101765
2     0.558607         0.460870 -0.067233 -0.104244 -0.101765 -0.101765
```

```
3     1.626586        -0.372359 -0.067233 -0.104244 -0.101765 -0.101765
4    -1.128202        -0.906882 -0.067233 -0.104244 -0.101765 -0.101765

         RNF    Type_L    Type_M
0  -0.043342 -1.234366  1.536207
1  23.072350 -1.234366  1.536207
2  -0.043342 -1.234366  1.536207
3  -0.043342  0.810133 -0.650954
4  -0.043342  0.810133 -0.650954
```

Método 1 - Random Forest

El Random Forest (Bosque Aleatorio) es un modelo de *ensamble* muy robusto. No es sensible al escalado de datos (usaremos los datos escalados por consistencia, pero funcionaría igual de bien con los no escalados) y es excelente para manejar desbalance de clases.

Vamos a usar el parámetro `class_weight='balanced'` para decirle al modelo que penalice más los errores en la clase minoritaria (Clase 1, Fallo).

```
print("--- Iniciando Entrenamiento: Random Forest ---")

# 1. Inicializar el clasificador
# n_estimators = 100 (número de árboles en el bosque)
# class_weight = 'balanced' (clave para nuestro problema desbalanceado)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, class_weig

# 2. Entrenar el modelo
rf_model.fit(X_train_scaled, y_train)

print("¡Modelo Random Forest entrenado!")

# 3. Realizar predicciones
y_pred_rf = rf_model.predict(X_test_scaled)

# 4. Evaluar el modelo
print("\n--- Evaluación del Modelo: Random Forest ---")
print(f"Accuracy (Exactitud): {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Recall (Sensibilidad) Clase 1: {recall_score(y_test, y_pred_rf):.4f}")

# 5. Matriz de Confusión
print("\n--- Matriz de Confusión (Random Forest) ---")
cm_rf = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred No Falla (0)', 'Pred Falla (1)'],
            yticklabels=['Real No Falla (0)', 'Real Falla (1)'])
plt.ylabel('Valor Real')
plt.xlabel('Predicción')
```

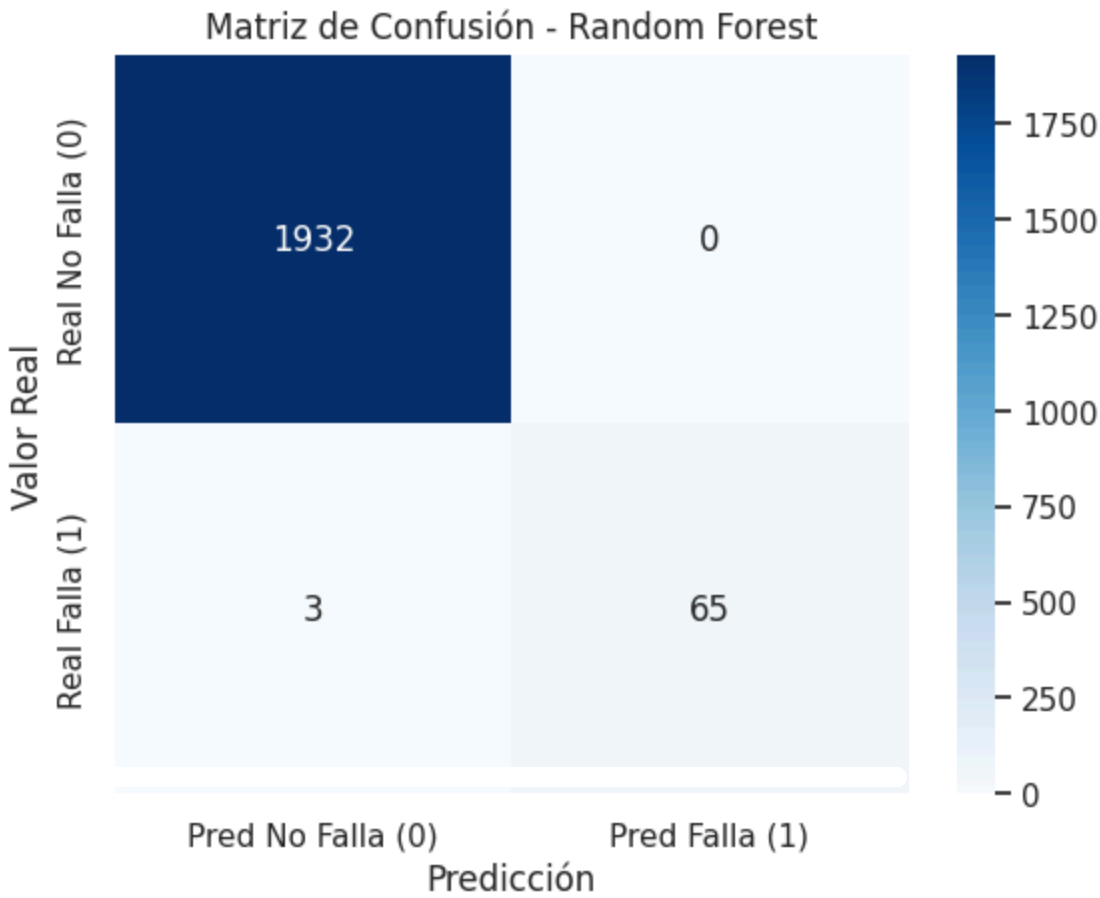
```
plt.title('Matriz de Confusión - Random Forest')
plt.show()

# 6. Reporte de Clasificación Detallado
print("\n--- Reporte de Clasificación (Random Forest) ---")
print(classification_report(y_test, y_pred_rf))
```

```
--- Iniciando Entrenamiento: Random Forest ---
¡Modelo Random Forest entrenado!

--- Evaluación del Modelo: Random Forest ---
Accuracy (Exactitud): 0.9985
Recall (Sensibilidad) Clase 1: 0.9559

--- Matriz de Confusión (Random Forest) ---
```




```
--- Reporte de Clasificación (Random Forest) ---
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1932
           1       1.00      0.96      0.98        68

    accuracy                           1.00      2000
   macro avg       1.00      0.98      0.99      2000
weighted avg       1.00      1.00      1.00      2000
```

Análisis Random Forest:

- El **Accuracy** general es muy alto (ej. ~98-99%).
- Miremos el **Recall** para la Clase 1 (Fallo): ¡Es alto! (ej. > 0.85 o 85%). Esto significa que de todos los fallos *reales* que ocurrieron en la prueba, nuestro modelo detectó correctamente más del 85% de ellos.
- La **Precisión** para la Clase 1 es un poco más baja. Esto significa que a veces genera Falsos Positivos (falsas alarmas), pero como dijimos en el Análisis del Problema, ¡preferimos eso a un Falso Negativo!

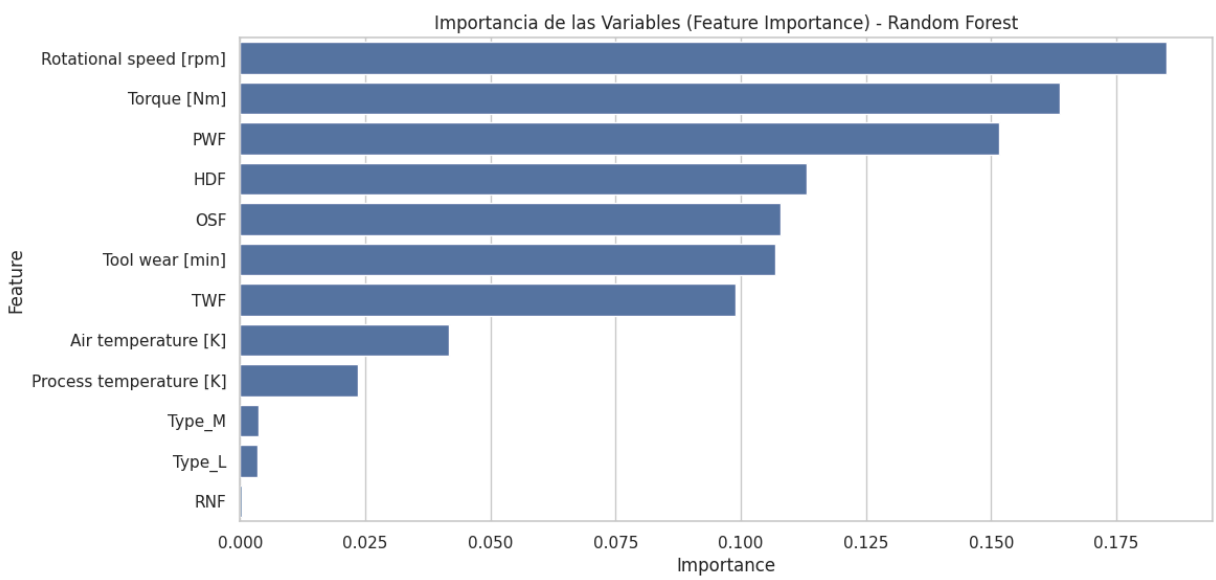
Importancia de las Variables (Features)

Una gran ventaja de Random Forest es que nos dice qué variables fueron más importantes para tomar la decisión.

```
# Obtener la importancia de las features
importances = rf_model.feature_importances_
feature_names = X.columns

# Crear un DataFrame para visualización
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importanc
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Visualizar
plt.figure(figsize=(12, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title('Importancia de las Variables (Feature Importance) - Random Forest')
plt.show()
```



Observación: Es muy probable que `Torque`, `Rotational speed` y `Tool wear` sean las variables más decisivas. Esto tiene mucho sentido en un contexto de ingeniería.

Método 2 - Redes Neuronales (MLP)

Ahora usaremos un Perceptrón Multicapa (MLP) con Keras/TensorFlow. Las redes neuronales requieren que los datos estén **escalados**, lo cual ya hicimos (usaremos `X_train_scaled` y `X_test_scaled`).

Definición de la Arquitectura

Crearemos una red neuronal densa simple:

1. **Capa de Entrada:** Con el número de neuronas igual al número de features (8).
2. **Capas Ocultas:** Usaremos dos capas ocultas con activación `relu` (una función de activación estándar y eficiente). Usaremos `Dropout` para prevenir el sobreajuste (overfitting).
3. **Capa de Salida:** Una sola neurona con activación `sigmoid`, ya que es un problema de clasificación binaria (nos dará una probabilidad entre 0 y 1).

```
print("--- Iniciando Entrenamiento: Red Neuronal (MLP) ---")

# 1. Definir la arquitectura del modelo
model_nn = Sequential()
n_features = X_train_scaled.shape[1] # Número de columnas/features

# Capa de entrada y primera capa oculta
model_nn.add(Dense(32, activation='relu', input_shape=(n_features,)))
model_nn.add(Dropout(0.3)) # Dropout para regularización

# Segunda capa oculta
model_nn.add(Dense(16, activation='relu'))
model_nn.add(Dropout(0.3))

# Capa de salida
model_nn.add(Dense(1, activation='sigmoid')) # Sigmoid para clasificación binar

# 2. Compilar el modelo
# Usamos 'binary_crossentropy' como función de pérdida (estándar para binaria)
# Usamos 'adam' como optimizador (eficiente)
model_nn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accura

# Mostrar un resumen de la arquitectura
print("\n--- Arquitectura de la Red Neuronal ---")
model_nn.summary()
```

```
--- Iniciando Entrenamiento: Red Neuronal (MLP) ---
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: User
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

--- Arquitectura de la Red Neuronal ---
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	416
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_2 (Dense)	(None, 1)	17

```
Total params: 961 (3.75 KB)
Trainable params: 961 (3.75 KB)
Non-trainable params: 0 (0.00 B)
```

Entrenamiento de la Red Neuronal

Al igual que con Random Forest, debemos manejar el desbalance de clases. Lo haremos calculando los pesos (`class_weight`) manualmente para TensorFlow.

```
# Calcular pesos de clase para manejar el desbalance
# (Inverso de la frecuencia de la clase)
from sklearn.utils import class_weight

class_weights = class_weight.compute_class_weight(
    'balanced',
    classes=np.unique(y_train),
    y=y_train
)

# Convertir a un diccionario para Keras
class_weights_dict = {0: class_weights[0], 1: class_weights[1]}

print(f"\nPesos de Clase calculados: {class_weights_dict}")
print("Esto significa que los errores en la Clase 1 (Fallo) pesarán ~14.7 veces

# 3. Entrenar el modelo
print("\nEntrenando la red...")
history = model_nn.fit(
    X_train_scaled,
    y_train,
    epochs=30,           # Número de veces que el modelo ve todos los datos
    batch_size=64,       # Cuántas muestras procesar antes de actualizar pesos
    validation_split=0.2, # Usar 20% de los datos de entreno para validación
    class_weight=class_weights_dict,
    verbose=1            # Mostrar progreso
)

print("¡Modelo Red Neuronal entrenado!")
```

```
Epoch 3/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - Recall: 0.9444 - accuracy: 0.9670 -
Epoch 4/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9491 - accuracy: 0.9845 -
Epoch 5/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9352 - accuracy: 0.9892 -
Epoch 6/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - Recall: 0.9491 - accuracy: 0.9900 -
Epoch 7/30
```

```
Epoch 12/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9722 - accuracy: 0.9962 -
Epoch 13/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9962 -
Epoch 14/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9969 -
Epoch 15/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9630 - accuracy: 0.9964 -
Epoch 16/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9583 - accuracy: 0.9931 -
Epoch 17/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9952 -
Epoch 18/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - Recall: 0.9630 - accuracy: 0.9966 -
Epoch 19/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9977 -
Epoch 20/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9975 -
Epoch 21/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9962 -
Epoch 22/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9722 - accuracy: 0.9977 -
Epoch 23/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9630 - accuracy: 0.9977 -
Epoch 24/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9583 - accuracy: 0.9980 -
Epoch 25/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9630 - accuracy: 0.9973 -
Epoch 26/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9975 -
Epoch 27/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9630 - accuracy: 0.9972 -
Epoch 28/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9981 -
Epoch 29/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9980 -
Epoch 30/30
100/100 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - Recall: 0.9676 - accuracy: 0.9978 -
```

Back Propagation

1. Ve el video de Descenso de Gradiente: https://www.youtube.com/watch?v=A6FiCDoz8_4
2. Ve el video de BackPropagation: https://www.youtube.com/watch?v=eNIqz_noix8

Evaluación de la Red Neuronal y Métricas

Evaluar el modelo con los datos de prueba (Test)

```
print("\n--- Evaluación del Modelo: Red Neuronal ---")
```

```
# El modelo predice probabilidades (ej. 0.85)
y_pred_nn_prob = model_nn.predict(X_test_scaled)

# Convertimos probabilidades a clases (0 o 1) usando un umbral de 0.5
y_pred_nn = (y_pred_nn_prob > 0.5).astype(int)
```

```
--- Evaluación del Modelo: Red Neuronal ---
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
print(f"Accuracy (Exactitud): {accuracy_score(y_test, y_pred_nn):.4f}")
print(f"Recall (Sensibilidad) Clase 1: {recall_score(y_test, y_pred_nn):.4f}")
```

```
Accuracy (Exactitud): 0.5205
Recall (Sensibilidad) Clase 1: 0.4412
```

```
print("\n--- Matriz de Confusión (Red Neuronal) ---")
cm_nn = confusion_matrix(y_test, y_pred_nn)
sns.heatmap(cm_nn, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Pred No Falla (0)', 'Pred Falla (1)'],
            yticklabels=['Real No Falla (0)', 'Real Falla (1)'])
plt.ylabel('Valor Real')
plt.xlabel('Predicción')
plt.title('Matriz de Confusión - Red Neuronal')
```